

A7

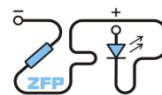
October 23, 2025

```
[1]: # TODO: Replace the string "Your Name" and "DD.MM.YYYY" with the appropriate ↵  
      ↵ values.
```

```
from header import header  
_ = header(student = "Jonáš Venc", date = "21. 10. 2025", task_no = 7)
```

Kabinet výuky obecné fyziky, UK MFF

Fyzikální praktikum 4



Úloha č. 7

Název úlohy: Pozitronová emisní tomografie.

Jméno: Jonáš Venc

Datum měření: 21. 10. 2025

Připomínky opravujícího:

Udělený počet bodů

Výsledky a zpracování měření	
Diskuse výsledků	
Závěr	
Seznam použité literatury	
Celkem	max. 12

Posuzoval:

dne:

1 Pozitronová emisní tomografie

Tato laboratorní úloha představuje zjednodušeným způsobem velmi účinnou metodu lékařského zobrazování. Pozitronová emisní tomografie (PET) se používá v lékařské diagnostice i v lékařském výzkumu v neurologii, onkologii, kardiologii a dalších oblastech medicíny.

PET lokalizuje místo vzniku gama fotonů v lidském těle. Tyto fotony vznikají v důsledku anihilace pozitronů a elektronů. Pozitrony jsou do těla zavedeny podáním radioaktivní látky a anihilují s elektrony atomů v tkáni. Konfigurace systému detekce fotonů umožňuje 3D rekonstrukci rozložení podané radioaktivní látky v těle. PET tak poskytuje obrazy tkání v závislosti na jejich schopnosti zachycovat tuto látku spíše než anatomické struktury.

Detektory jsou uspořádány v kruhu obklopujícím pacienta a registrují vylétávající anihilační fotony. Detektory jsou vybaveny koincidenční elektronikou, takže jsou zaznamenávány pouze dvojice fotonů přiletávajících ve stejném čase. Každý pár fotonů pak může být použit k sestavení přímky procházející tělem. Použitím velkého počtu takových přímek lze rekonstruovat distribuci pozitronových zdrojů v objemu těla.

Izotop	^{11}C	^{13}N	^{15}O	^{18}F
Poločas	20 min	10 min	2 min	110 min

Nejčastější radionuklidy pro PET jsou začleněny do běžných metabolitů, jako je glukóza nebo voda, a podávány pacientovi. Radionuklidy je také možné začlenit do složitějších molekul, jako jsou léky, a použít PET ke sledování cesty léku v těle.

1.0.1 Anihilace pozitronu

Pozitron je antičástice k elektronu. Narozdíl od elektronu se v přírodě vyskytuje jen vzácně, protože rychle anihiluje s elektronem. Během anihilace obě částice zmizí a vzniknou dva anihilační fotony. Pozitrony jsou emitovány při β^+ rozpadu některých radioizotopů. Při průchodu hmotou pozitrony nejprve ztratí svou kinetickou energii rozptylem, převážně na elektronech v okolní hmotě. Když je kinetická energie pozitronu blízká nule, anihiluje s jedním z blízkých elektronů. Obvykle trvá $10^{-10} - 10^{-7}$ s, než emitovaný pozitron anihiluje, a během této doby urazí nanejvýš několik desetin milimetru. Například ^{22}Na emituje pozitrony s energií nanejvýš 0,8 MeV a pozitron s touto energií urazí ve vodě maximálně 1 milimetr. Při anihilaci elektron-pozitron vznikají dva fotony. V případě, že pozitron i elektron mají kinetickou energii blízkou nule, je energie anihilačních fotonů rovna součtu klidových hmotností pozitronu a elektronu. Zákon zachování hybnosti vyžaduje, aby celková hybnost fotonů byla prakticky nulová, a proto musí mít stejně velké, ale opačně orientované hybnosti. Velikost hybnosti každého fotonu je 511 keV/c. Střední volná dráha takových fotonů v tkáni je několik centimetrů a ve vodě 7,2 cm.

1.0.2 Koincidenční měření

Fyzikální jevy prezentované v této úloze lze studovat koincidenčním měřením. Toto měření je založeno na elektronickém koincidenčním bloku, který vydává signál pouze tehdy, když na vstupy [3] dorazí současně alespoň dva impulsy. Koincidence fyzikálních jevů je rozmazána kvůli fluktuacím tvorby signálu v detektorech a časovému rozlišení koincidenční elektroniky. Časové rozlišení pro koincidenční měření dvou nebo více současných jevů závisí především na rychlosti tvorby signálu v

detektorech a u nejmodernější elektroniky dosahuje 10^{-6} až 10^{-12} s. Koincidence vznikající detekcí obou fotonů z jediné anihilace nazýváme *skutečné koincidence*. Pokud dva pozitrony anihilují náhodou v rámci koincidenčního časového okna, můžeme detekovat koincidence vznikající v důsledku dvou různých anihilací. Tyto koincidence se nazývají *falešné*. Četnost falešných koincidencí N_s lze vypočítat pomocí (nekorelovaných) četností signálu na dvou detektorech, N_1 a N_2 , a šířky koincidenčního časového okna τ :

$$N_s = 2N_1N_2\tau$$

Pokud máme více detektorů, bude výsledný vztah pro četnost falešných koincidencí vypadat takto:

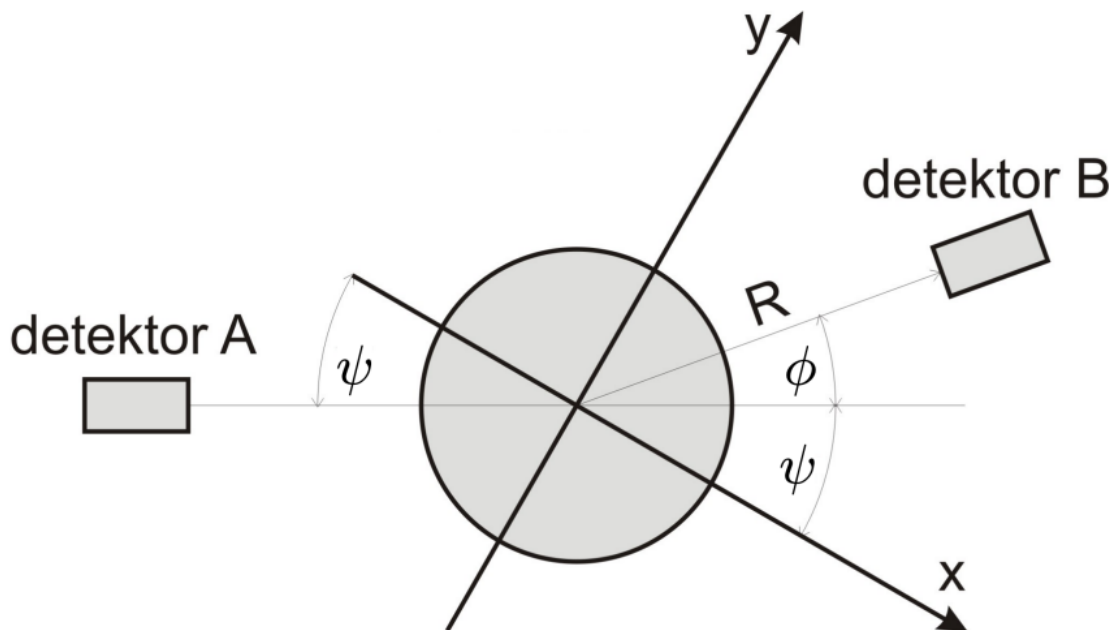
$$N_s = 2^{n-1}N_1N_2 \dots N_n\tau^{n-1}$$

1.0.3 Uspořádání experimentu

Obr. 1 ukazuje uspořádání experimentu a používané úhlové proměnné. Zdroje pozitronů (^{22}Na) jsou umístěny v plastovém držáku ve tvaru lebky, kterým se dá otáčet. Úhel otočení lebky vzhledem ke stolu se měří úhloměrem pro plný rozsah 360 deg a pevnou šipkou označující směr na stole. Dva Na(Tl) scintilační detektory A a B se používají k detekci anihilačních fotonů.

Detektory mají konečné rozměry a konečné úhlové rozlišení - mohou detekovat fotony přicházející pod úhlem k ose detektoru. Úhlový interval, ve kterém dochází k registraci fotonu se nazývá *úhlové rozlišení* a omezuje přesnost měření směru fotonů. Signály z obou detektorů jsou - po zesílení a diskriminaci - přiváděny do koincidenčního bloku a dále do koincidenčního počítače.

Obr. 1 Pozitronová emisní tomografie: uspořádání experimentu a používané úhlové proměnné.



1.0.4 Geometrie experimentu: souřadné systémy

Obr. 1 znázorňuje dva souřadné systémy použité v našem experimentu. První je polární systém souřadnic spojený se stolem. Jeho počátek se shoduje s osou otáčení lebky, a zároveň s osou otáčení

ramene, na kterém je upevněný detektor B. V tomto systému jsou měřené souřadnice detektorů A a B. Detektor A je nepohyblivý, a tedy má pevné souřadnice $(R, 180 \text{ deg})$. Detektor B umístěný na otočném rameni délky R má souřadnice (R, ϕ) . Druhý souřadný systém je kartézský a je spojen s modelem lebky. Jeho počátek se shoduje s osou otáčení lebky. Cílem této úlohy je určit polohu dvou zdrojů pozitronů vzhledem k této soustavě. Souřadnice detektorů A a B v kartézské soustavě modelu otočené o ψ vzhledem ke stolu jsou

$$x_A = -R \cos \psi \quad x_B = R \cos (\psi + \phi)$$

$$y_A = -R \sin \psi \quad y_B = R \sin (\psi + \phi)$$

Rovnice přímky spojující detektory A a B v kartézské soustavě spojené s lebkou je

$$(y_B - y_A)x - (x_B - x_A)y = (y_B - y_A)x_A - (x_B - x_A)y_A.$$

1.0.5 Data lebky

Nejprve připravíme data pro obrázek lebky, který použijeme ke zobrazení našich výsledků.

Úkol: Spusťte následující buňku pro načtení dat lebky.

```
[2]: # Solution.

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib as mpl
import seaborn as sns

# Set the notebook directory. If you store the data in a different directory
# than this notebook, change the path accordingly.
A7_home = "."

# Read skull data from file for future use
data_x = []
data_y = []
with open(A7_home + "/skull_x_y.txt", "r") as infile:
    for line in infile:
        x, y = [float(s) for s in line.split()]
        data_x.append(x)
        data_y.append(y)
skull_data = pd.DataFrame({"x": data_x, "y": data_y})
```

2 Měření s jediným pozitronovým zdrojem

V tomto pracovním úkolu předpokládáme, že anihilační fotony vznikají v jediném bodovém zdroji a vyletují v přesně opačných směrech. V takovém případě pozorujeme skutečné koincidence, pokud se zdroj nachází na přímce spojující detektory A a B.

2.0.1 Postup měření

Instruktor umístí jeden zdroj ^{22}Na do modelu lebky. Poté otočte model lebky na $\psi_1 = 0$ deg a změřte rozdělení četnosti koincidencí v závislosti na úhlu ϕ . Poté otočte lebku na $\psi_2 = 90$ deg a změřte rozdělení četnosti koincidencí opět v závislosti na ϕ . V obou případech měřte v dostatečně širokém intervalu úhlů ϕ . Polohy maxim těchto dvou rozdělení, ϕ_1 a ϕ_2 , použijeme pro konstrukci přímek procházejících zdrojem. Průsečík těchto přímek určí polohu zdroje v modelu lebky.

Akumulační čas nastavte pro každé měření četnosti na 20 s.

Formát dat musí být následující:

- text sestávající ze tří sloupců oddělených jedním, nebo několika prázdnými znaky (mezera, tabulátor)
- první sloupec: úhel ψ ve stupních
- druhý sloupec: úhel ϕ ve stupních
- třetí sloupec: počet koincidencí N zaznamenaný během akumulčního času 20 s
- první řádek musí obsahovat záhlaví sloupců **psi**, **phi** a **N** oddělené jedním, nebo několika prázdnými znaky (mezera, tabulátor); použijte přesně tato jména sloupců

Úkol: Změřte dvě úhlová rozdělení popsaná výše. Naměřená data pište do stringu `data_str` níže.

```
[3]: # Solution.

# TODO: Fill the string below with your data.
# The string will look like the following, but with many more lines, of course:
# data_str = """
# psi phi N
# 0 -35 5
# 0 -31 11
# 90 -35 6
# 90 -31 10
# """

data_str = """
psi phi N
90 -40 4
90 -35 5
90 -30 3
90 -25 6
90 -20 6
90 -15 4
90 -10 11
90 -7 23
90 -4 44
90 -2 54
90 0 77
90 1 93
```

```
90 2 107
90 3 108
90 4 120
90 5 136
90 6 136
90 7 125
90 8 161
90 9 160
90 10 142
90 11 137
90 12 147
90 13 168
90 14 141
90 15 137
90 16 136
90 17 111
90 18 110
90 19 111
90 20 93
90 22 91
90 24 69
90 27 27
90 30 19
90 35 6
90 40 8
0 0 3
0 10 24
0 13 47
0 15 70
0 16 78
0 17 105
0 18 114
0 19 149
0 20 144
0 21 136
0 22 149
0 23 164
0 24 168
0 25 139
0 26 161
0 27 125
0 28 99
0 30 88
0 33 51
0 35 20
0 40 10
" " " "
```

Úkol: Ze stringu obsahujícího vaše data vytvořte pandas DataFrame `data_1`. Do něj přidejte také sloupec `err`, který bude obsahovat neurčitosti počtu koincidencí, které lze odhadnout jako $\sqrt{N}$. DataFrame `data_1` vypište jako tabulku.

```
[4]: # Solution.

from io import StringIO

# Create the pandas dataframe from the string above.
data_1 = pd.read_table(StringIO(data_str), sep = "\s+")

# Add a column with uncertainties in counts.
data_1["err"] = np.sqrt(data_1["N"])

# Print data_1 as a table.
print(data_1.rename(columns = {"psi": "psi [deg]", "phi": "phi [deg]", "N": "N", "err": "N_unc"}).to_string(index = False, max_rows = None))

# # ----- SAFE TO IGNORE BELOW THIS LINE -----
# # ALTERNATIVE SOLUTION: Read the data from a file.
# # The file has the same format as the `data_str` string above: three columns
# # with a header line. The columns are separated by whitespace (spaces or
# # tabs).
# # The header line is: "psi phi N".

# # Path to the data file.
# data_file = A7_home + "/sample_data/set_A/Task1.txt"

# # Read the data of a single source angular profile from a file and convert to
# # a pandas dataframe
# data_1 = pd.read_table(data_file, sep = "\s+")
# print(data_1.head())
# data_1["err"] = np.sqrt(data_1["N"]) # compute uncertainties in counts
```

psi [deg]	phi [deg]	N	N_unc
90	-40	4	2.000000
90	-35	5	2.236068
90	-30	3	1.732051
90	-25	6	2.449490
90	-20	6	2.449490
90	-15	4	2.000000
90	-10	11	3.316625
90	-7	23	4.795832
90	-4	44	6.633250
90	-2	54	7.348469
90	0	77	8.774964
90	1	93	9.643651

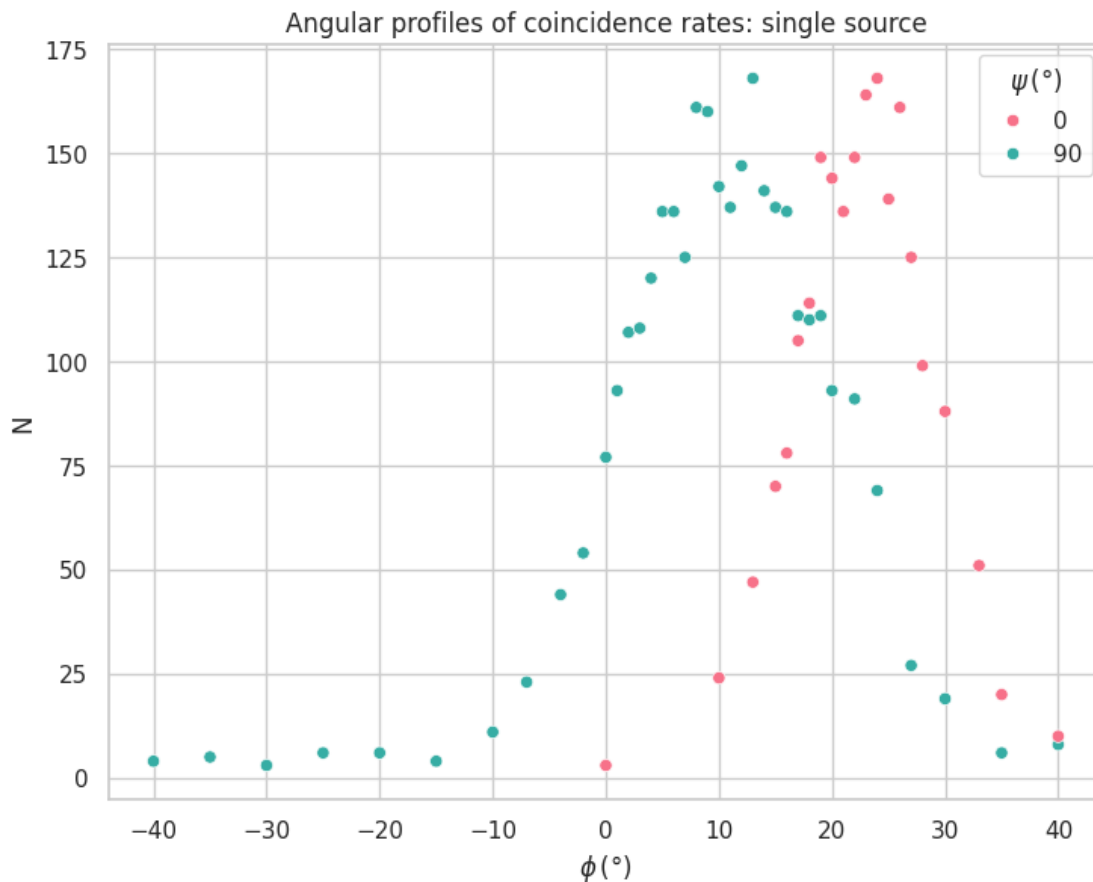
90	2	107	10.344080
90	3	108	10.392305
90	4	120	10.954451
90	5	136	11.661904
90	6	136	11.661904
90	7	125	11.180340
90	8	161	12.688578
90	9	160	12.649111
90	10	142	11.916375
90	11	137	11.704700
90	12	147	12.124356
90	13	168	12.961481
90	14	141	11.874342
90	15	137	11.704700
90	16	136	11.661904
90	17	111	10.535654
90	18	110	10.488088
90	19	111	10.535654
90	20	93	9.643651
90	22	91	9.539392
90	24	69	8.306624
90	27	27	5.196152
90	30	19	4.358899
90	35	6	2.449490
90	40	8	2.828427
0	0	3	1.732051
0	10	24	4.898979
0	13	47	6.855655
0	15	70	8.366600
0	16	78	8.831761
0	17	105	10.246951
0	18	114	10.677078
0	19	149	12.206556
0	20	144	12.000000
0	21	136	11.661904
0	22	149	12.206556
0	23	164	12.806248
0	24	168	12.961481
0	25	139	11.789826
0	26	161	12.688578
0	27	125	11.180340
0	28	99	9.949874
0	30	88	9.380832
0	33	51	7.141428
0	35	20	4.472136
0	40	10	3.162278

Úkol: Vykreslete naměřená data jako bodový graf četnosti koincidencí N v závislosti na úhlu ϕ

pro oba úhly ψ .

```
[5]: # Solution.  
  
# Read the psi values that were used in the experiment.  
psi_vals = data_1["psi"].unique()  
  
# Plot the number of counts vs. phi for all psi values.  
sns.set_theme(style = "whitegrid")  
fig, ax = plt.subplots(figsize=(8.5, 6.5))  
sns.scatterplot(data = data_1, x = "phi", y = "N", hue = "psi", palette = sns.  
    color_palette("husl", len(psi_vals)), ax = ax)  
plt.xlabel("$\\phi\\$, (\\degree)$")  
plt.legend(title = "$\\psi\\$, (\\degree)$")  
plt.title("Angular profiles of coincidence rates: single source")
```

```
[5]: Text(0.5, 1.0, 'Angular profiles of coincidence rates: single source')
```



Úkol: Vysvětlete tvar naměřených úhlových rozdělení počtu koincidenčí.

Řešení:

V ideálním případě bodového detektoru bychom očekávali tvar podobný delta funkci (s vysokou hodnotou, ne nekonečnou). Pro náš detektor pozorujeme rozdělení s určitým peakem, jehož poloha odpovídá detekci pravých fotonů, tedy fotonů vylétávajících se vzájemným úhlem 180° a tedy takové poloze detektorů. Protože detektory nejsou bodové, ale mají určitou šířkou pozorujeme i nižší hodnoty koincidence pro určité odchylky úhlů. Mimo tyto úhly neměříme nulovou incidenci, ale určité pozadí viz. další diskuze.

2.0.2 Fitování dat

Nyní určíme polohy maxim naměřených úhlových rozdělení. Na každé úhlové rozdělení nafitujeme funkci, která je součtem normálního rozdělení a konstanty:

$$f(\phi) = bg + amp \cdot \exp \left[-\frac{1}{2} \left(\frac{\phi - mean}{width} \right)^2 \right]$$

Nafitované hodnoty parametrů `mean` použijeme jako náš odhad poloh maxim úhlových rozdělení.

Úkol: Úhlové rozdělení pro každou hodnotu ψ nafitujte výše uvedenou funkcí. Nafitované parametry a jejich neurčitosti uveďte v tabulce.

```
[6]: # Solution.

# Fit the data.
# Store the fit results in a dictionary `fits`.

from scipy.optimize import curve_fit

def gauss_plus_background(x, bg, amp, mean, w):
    """
    Function returning a gaussian with flat background,
    Meaning of parameters is clear from the definition.
    """
    return bg + amp * np.exp(- 1/2 * ((x-mean)/w)**2)

psi_vals = data_1["psi"].unique() # psi values used
fits = dict.fromkeys(psi_vals)
for psi in psi_vals:
    data_psi = data_1[data_1["psi"] == psi]
    p_opt, p_cov = curve_fit(gauss_plus_background, data_psi["phi"].values,
    ↪ data_psi["N"].values, p0 = [0, max(data_psi["N"]), 24, 5],
                                sigma = data_psi["err"].values,
    ↪ absolute_sigma = True)
    p_err = np.sqrt(np.diag(p_cov))
    errXerr = np.outer(p_err, p_err)
    p_corr = p_cov / errXerr
    fits[psi] = {"parameters": p_opt, "par_errors": p_err, "correlations" :
    ↪ p_corr}
```

```

# Print a table of results.
# For the rendering to pdf, split the table in two.
yields_table = pd.DataFrame({
    "psi (deg)" : psi_vals,
    "bg" : [fits[psi]["parameters"][0] for psi in psi_vals],
    "s(bg)" : [fits[psi]["par_errors"][0] for psi in psi_vals],
    "amp" : [fits[psi]["parameters"][1] for psi in psi_vals],
    "s(amp)" : [fits[psi]["par_errors"][1] for psi in psi_vals],
})

fits_table = pd.DataFrame({
    "psi (deg)" : psi_vals,
    "mean (deg)" : [fits[psi]["parameters"][2] for psi in psi_vals],
    "s(mean) (deg)" : [fits[psi]["par_errors"][2] for psi in psi_vals],
    "width (deg)" : [fits[psi]["parameters"][3] for psi in psi_vals],
    "s(width) (deg)" : [fits[psi]["par_errors"][3] for psi in psi_vals]
})

print("Parameters of Gaussian fits to profiles of coincidence rates")
print(yields_table.to_string(index = False, float_format = "{:.2f}".format))
print(fits_table.to_string(index = False, float_format = "{:.2f}".format))

```

```

Parameters of Gaussian fits to profiles of coincidence rates
psi (deg)  bg  s(bg)  amp  s(amp)
      90 3.85  0.83 151.17  3.52
      0 3.58  1.61 154.30  4.50
psi (deg)  mean (deg)  s(mean) (deg)  width (deg)  s(width) (deg)
      90          10.87          0.22          8.95          0.23
      0          22.87          0.19          6.16          0.20

```

Úkol: Do DataFrame `data_1` přidejte:

- sloupec `N_fit`, který bude obsahovat hodnoty fitované funkce v bodech ϕ ,
- sloupec `residual`, který bude obsahovat veličinu $(N - N_{\text{fit}})/\sqrt{N_{\text{fit}}}$, tzv. reziduály.

Do dictionary `fits` přidejte hodnotu χ^2 , `chisq` a počet stupňů volnosti, `df`, pro každý fit.

[7]: *# Solution.*

```

def add_fitted_to_data(data):
    """
    Adds fitted values and residuals to data points.
    Also adds chi2 and df to the fits dictionary.
    """
    result = data.copy()
    psi = np.mean(result["psi"])
    parameters = fits[psi]["parameters"]
    result["N_fit"] = gauss_plus_background(data["phi"], *parameters)

```

```

    result["residual"] = (result["N"] - result["N_fit"]) / np.
    ↪sqrt(result["N_fit"])
    fits[psi]["chisq"] = sum(r**2 for r in result["residual"]) # Side effect!
    nrow, ncol = data.shape
    fits[psi]["df"] = nrow - 4
    return result

# Apply the function add_fitted_to_data.
data_1 = data_1.groupby("psi")[["psi", "phi", "N", "err"]].
    ↪apply(add_fitted_to_data)
data_1.index = data_1.index.droplevel("psi")

```

Úkol: Naměřená data a nafitované funkce zobrazte v jednom grafu.

```

[8]: # Solution.

def make_fine_plot(data):
    '''
    Data to draw a fitted function plot
    '''
    psi = np.mean(data["psi"])
    parameters = fits[psi]["parameters"]
    x = np.linspace(min(data["phi"]), max(data["phi"]), 100)
    y = gauss_plus_background(x, *parameters)
    return pd.DataFrame({"psi": np.repeat(psi, 100), "phi": x, "N_fit": y})

fine_data = data_1.groupby("psi")[["psi", "phi", "N_fit"]].apply(make_fine_plot)

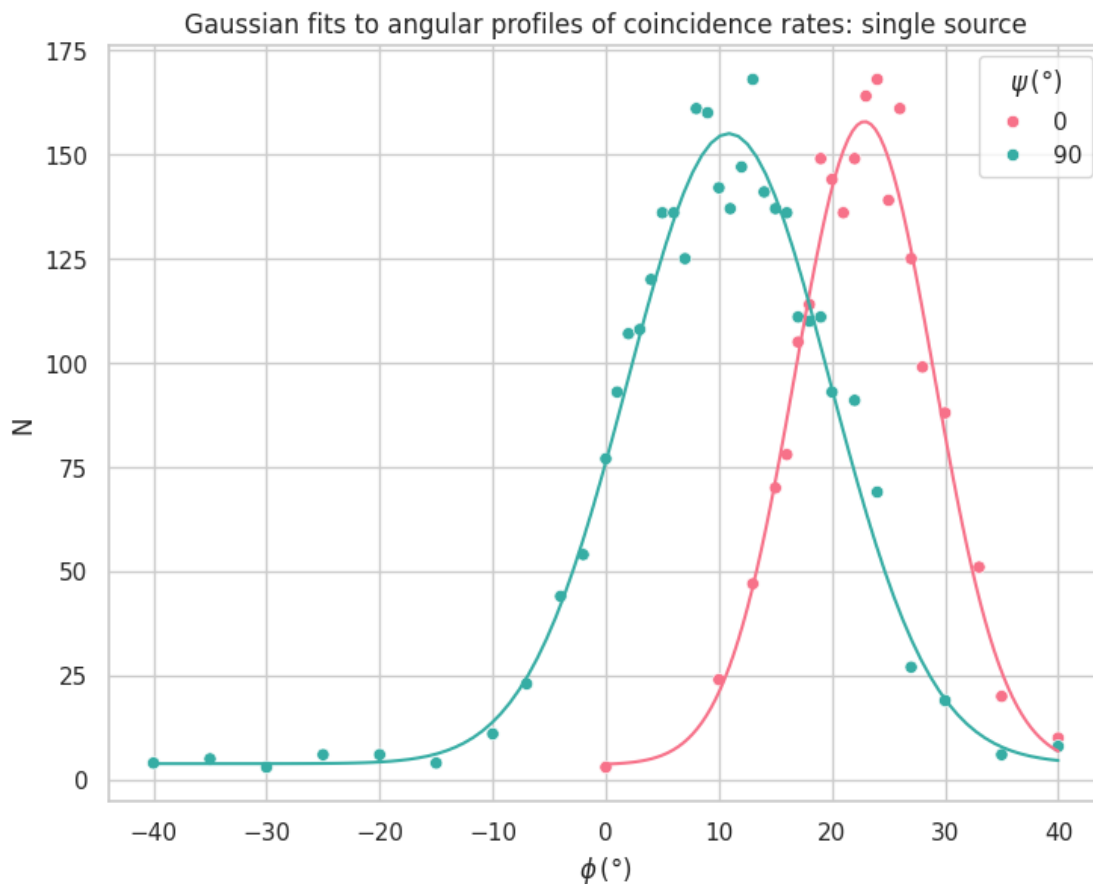
sns.set_theme(style = "whitegrid")
fig, ax = plt.subplots(figsize=(8.5, 6.5))
sns.scatterplot(data = data_1, x = "phi", y = "N", hue = "psi", palette = sns.
    ↪color_palette("husl", len(psi_vals)), ax = ax)
sns.lineplot(data = fine_data, x = "phi", y = "N_fit", hue = "psi", palette =
    ↪sns.color_palette("husl", len(psi_vals)), ax = ax, legend = False)
plt.xlabel("$\\phi$, (\\degree)$")
plt.legend(title = "$\\psi$, (\\degree)$")
plt.title("Gaussian fits to angular profiles of coincidence rates: single_
    ↪source")

```

```

[8]: Text(0.5, 1.0, 'Gaussian fits to angular profiles of coincidence rates: single
source')

```



2.0.3 Diagnostické grafy

Nyní se budeme zabývat otázkou, zda je náš fit rozumný - zda zvolený model odpovídá naměřeným datům.

Úkol: Pro každý fit vypište hodnoty χ^2 , počet stupňů volnosti a redukovaný χ^2 .

[9]: *# Solution.*

```
# Print the chi2 and df values.
for psi in data_1["psi"].unique():
    chi2 = fits[psi]["chisq"]
    df = fits[psi]["df"]
    print(f"psi = {psi:3.0f} deg: chi2 = {chi2:6.1f}, df = {df:2d}, chi2/df = {chi2/df:5.2f}")
```

```
psi = 0 deg: chi2 = 17.6, df = 17, chi2/df = 1.04
```

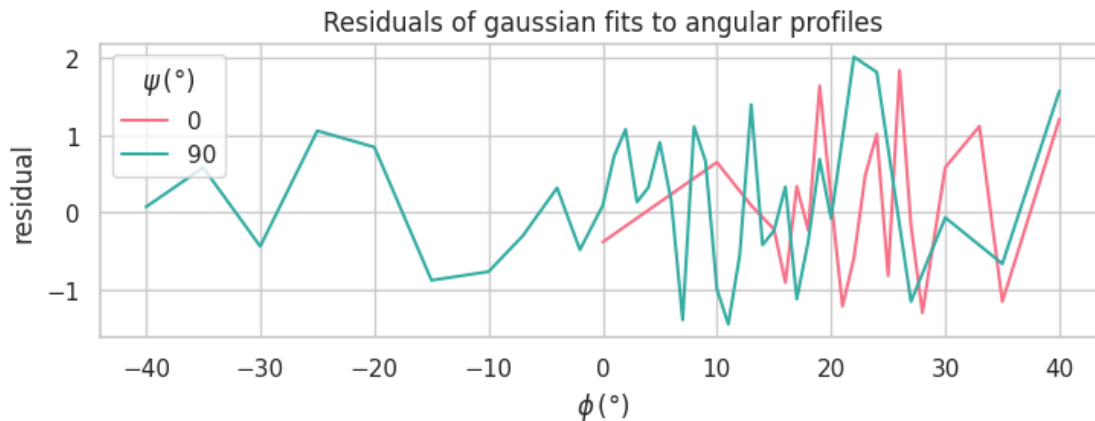
```
psi = 90 deg: chi2 = 29.7, df = 33, chi2/df = 0.90
```

Úkol: Do jednoho grafu nakreslete reziduály pro každé úhlové rozdělení.

```
[10]: # Solution.

fig, ax = plt.subplots(figsize = [8.5, 2.5])
sns.lineplot(data = data_1, x = "phi", y = "residual", hue = "psi", palette = "husl",
             dashes = True, ax = ax)
plt.xlabel("$\\phi$, (\\degree)$")
plt.legend(title = "$\\psi$, (\\degree)$")
plt.title("Residuals of gaussian fits to angular profiles")
```

[10]: Text(0.5, 1.0, 'Residuals of gaussian fits to angular profiles')



Úkol: Pro každé úhlové rozdělení nakreslete tzv. Quantile-Quantile (QQ) graf. Opět je cílem zjistit, jestli fitovaná funkce dobře popisuje naměřená data.

```
[11]: # Solution.

from scipy.stats import norm

def make_QQ_data(data):
    psi = np.mean(data["psi"]).astype(int)
    y = np.sort(data["residual"])
    mean, std = np.mean(y), np.std(y)
    ppf = norm(loc=mean, scale=std).ppf # Inverse CDF
    N = len(y)
    x = [ppf(i / (N+2)) for i in range(1,N+1)]
    nrow, ncol = data.shape
    return pd.DataFrame({"psi": np.repeat(psi, nrow), "x": x, "y": y})

qq_data = data_1.groupby("psi")[["psi", "residual"]].apply(make_QQ_data)

fig, ax = plt.subplots(figsize = [8.5, 8.5])
```

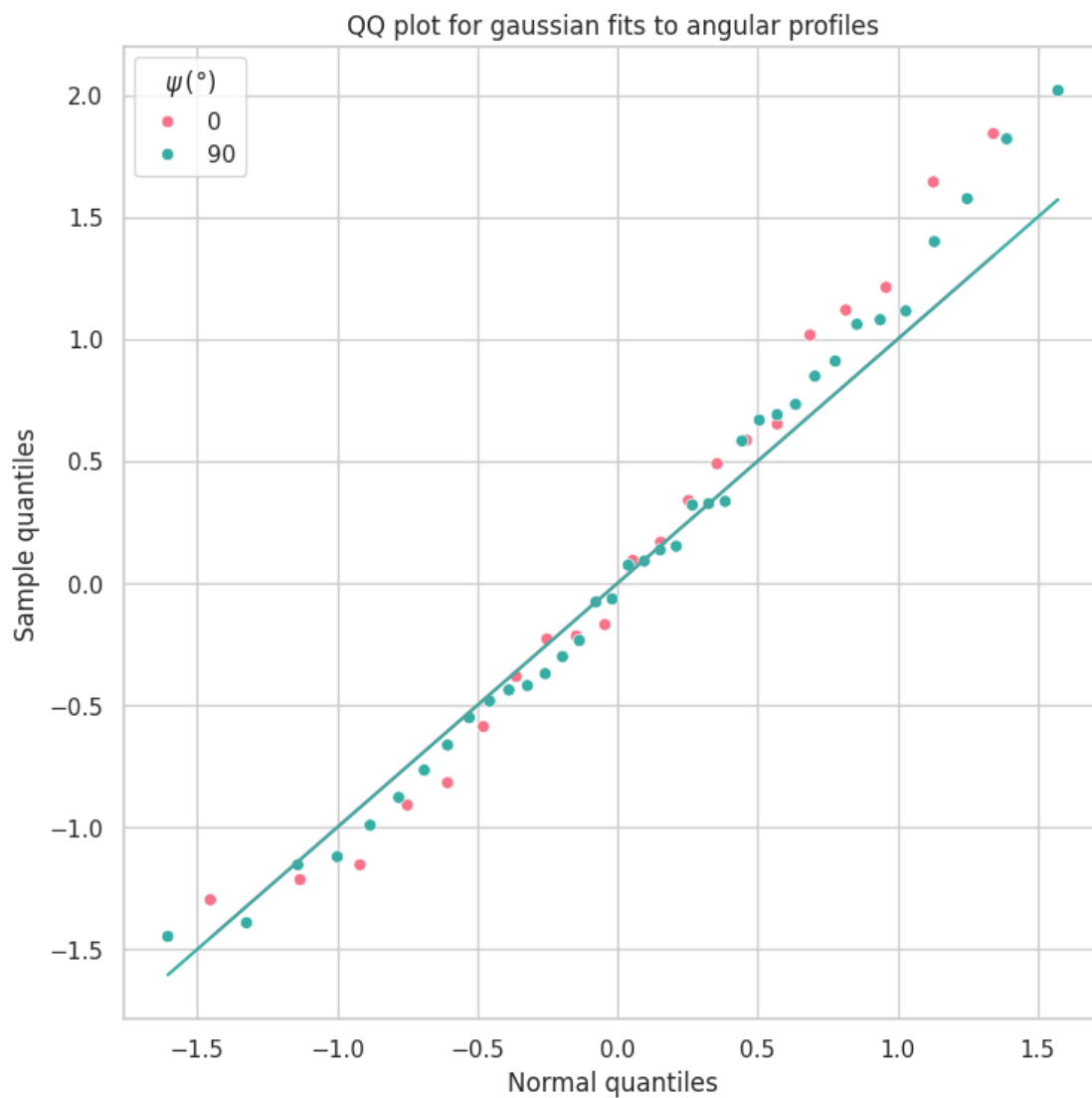
```

sns.scatterplot(data = qq_data, x = "x", y = "y", hue = "psi", palette = sns.
    ↳color_palette("husl", len(psi_vals)), ax = ax)
sns.lineplot(data = qq_data, x = "x", y = "x", hue = "psi", palette = sns.
    ↳color_palette("husl", len(psi_vals)), ax = ax, legend = False)

# Add labels
plt.xlabel('Normal quantiles')
plt.ylabel('Sample quantiles')
plt.legend(title = "$\\psi\\$, (\\degree)$")
plt.title("QQ plot for gaussian fits to angular profiles")

```

[11]: Text(0.5, 1.0, 'QQ plot for gaussian fits to angular profiles')



Úkol: Diskutujte kvalitu fitu a tvar naměřených úhlových rozdělení. Odpovězte mj. na následující otázky:

- Je provedený fit dobrý? Mohl by být vylepšen?
- Jsou profily gaussovské, nebo existuje podstatná odchylka od gaussovskosti? Proč?

Hint: dobrý fit by měl splňovat následující kritéria:

- Redukovaný χ^2 by měl být v rozumné blízkosti čísla 1.
- 95 % normalizovaných reziduálů by mělo být mezi -2 a 2.
- QQ graf by měl být přibližně lineární.

Řešení:

V datech měření máme uloženou jednu hodnotu pro úhel $\psi = 0^\circ$ a $\phi = 5$ počet incidencí 0. Jedná se téměř určitě o hrubou chybu, proto jsme ji z datasetu vyřadili. Navíc pokud bych ji ponechali, dostali bychom chybové hlášky z výpočtů, kde se dělí nejistotou incidence (což je odmocnina z počtu, tedy 0). Pravděpodobně se jedná jednoduše o špatně opsaný údaj.

Dále jsme zoptimalizovali nastavení původního odhadu funkce p_0 tak, aby fitovaná křivka co nejvíce odpovídala. Po těchto vylepšeních jsme dosáhli dobrého fitu, což potvrzuje QQ graf.

Zvýšit přesnosti bychom mohli zvýšením měřicího intervalu, v našem případě jsme měřili 20 s. Významněji bychom přesnost mohli zvýšit navýšením počtu měřených bodů (úhlů) s vyšší přesností a tedy menším krokem - alespoň 0.5° . Drobným vylepšením by mohlo být zvětšením vzdálenosti mezi detektory a tím snížení náhodné incidence. Bylo by také možné určitým způsobem odečíst pozadí, to však ale nemá vliv na parametry pro nás důležité.

Podle všech výše uvedených kritérií můžeme fit označit z dobrý a dobře odpovídá gaussovskému profilu s rozdílem nulovosti na okrajích (gaussova křivka klesá do nuly).

2.0.4 Výpočet polohy pozitronového zdroje

Z naměřených úhlových rozdělení jsme pomocí fitu určili polohy maxim, ϕ_1 a ϕ_2 . Máme tedy dvě dvojice (ψ_1, ϕ_1) a (ψ_2, ϕ_2) , z nichž dostaneme rovnice dvou přímek. Obě tyto přímky spojují detektor A, zdroj pozitronů a detektor B.

Nejprve si připomeňme, že rovnice každé přímky spojující detektory A a B v kartézské soustavě spojené s lebkou je

$$(y_B - y_A)x - (x_B - x_A)y = (y_B - y_A)x_A - (x_B - x_A)y_A,$$

kde souřadnice detektorů A a B jsou dány vztahem

$$x_A = -R \cos \psi \quad x_B = R \cos (\psi + \phi)$$

$$y_A = -R \sin \psi \quad y_B = R \sin (\psi + \phi)$$

Úkol: Pro každou dvojici (ψ_1, ϕ_1) a (ψ_2, ϕ_2) vypočítejte polohy detektorů A a B v kartézské soustavě spojené s lebkou. Výsledky vypište v tabulce.

```

[12]: # Solution.

from uncertainties import *
import uncertainties.umath as umath

# Distance from the center of the skull to the detectors.
R = ufloat(15, 0.5) # We are not sure about the correct value, but the
    ↪ uncertainty translates directly to uncertainties in estimated positions

def to_xy(R, psi, phi):
    """
    For given angle positions of the skull and the movable detector arm gives
    xy positions of centers of detector A and detector B.
    """
    rad = 3.1415926 / 180.0
    return -R * umath.cos(psi * rad), \
        -R * umath.sin(psi * rad), \
        R * umath.cos(psi * rad + phi * rad), \
        R * umath.sin(psi * rad + phi * rad)

# -----
# Convert fit parameters to random variables and set their correlations.
# This will help us propagate parameter uncertainties through calculations.
# -----

rv_fits = {} # contains line points as random variables
par_names = ["bg", "amp", "mean", "std"]
for psi, fit in fits.items():
    rv_list = []
    for par, err in zip(fit["parameters"], fit["par_errors"]):
        rv_list.append((par, err))
    rv_list = correlated_values_norm(rv_list, fit["correlations"])
    rv_fits[psi] = {par_names[i]: rv_list[i] for i in range(len(rv_list))}

# -----
# Get the positions of detectors A and B.
# -----

points = {} # collecting random variables
points_records = [] # collecting for table

for psi, par in rv_fits.items():
    ax, ay, bx, by = to_xy(R, psi, par["mean"])
    points[psi] = {"A": (ax, ay), "B": (bx, by)}
    points_records.append({
        "psi (deg)" : psi,

```

```

        "AB" : "A",
        "x (cm)" : ax.nominal_value,
        "s(x) (cm)" : ax.std_dev,
        "y (cm)" : ay.nominal_value,
        "s(y) (cm)" : ay.std_dev,
    })
    points_records.append({
        "psi (deg)" : psi,
        "AB" : "B",
        "x (cm)" : bx.nominal_value,
        "s(x) (cm)" : bx.std_dev,
        "y (cm)" : by.nominal_value,
        "s(y) (cm)" : by.std_dev,
    })

points_table = pd.DataFrame.from_records(points_records)
print("Estimated positions of detectors A and B")
print(points_table.to_string(index = False, float_format = "{:.2f}".format))

```

Estimated positions of detectors A and B

psi (deg)	AB	x (cm)	s(x) (cm)	y (cm)	s(y) (cm)
90	A	-0.00	0.00	-15.00	0.50
90	B	-2.83	0.11	14.73	0.49
0	A	-15.00	0.50	-0.00	0.00
0	B	13.82	0.46	5.83	0.20

Nyní snadno odvodíme rovnice obou přímek. Pro jednoduchost hledání jejich průsečíku si koeficienty vystupující v obou přímkách přeznačíme následujícím způsobem:

$$a_{11}x + a_{12}y = b_1,$$

$$a_{21}x + a_{22}y = b_2.$$

Souřadnice průsečíku, (x, y) , těchto dvou přímek můžeme najít řešením soustavy například pomocí Cramerova pravidla. Toto řešení je odhadem polohy zdroje pozitronů v souřadnicovém systému lebky.

Úkol: Zkonstruuje rovnice obou přímek a najděte jejich průsečík. Souřadnice průsečíku a jejich neurčitosti vypište.

```

[13]: # Solution.

# X-points of pairs of lines: size-2 combinations of all psi measurements
from itertools import combinations

X_points = [] # for random variables
rows = [] # for a table
for psi1, psi2 in combinations(psi_vals, 2):
    ax1, ay1 = points[psi1]["A"]
    bx1, by1 = points[psi1]["B"]

```

```

ax2, ay2 = points[psi2]["A"]
bx2, by2 = points[psi2]["B"]
a = by1 - ay1
b = - bx1 + ax1
c = a * ax1 + b * ay1
d = by2 - ay2
e = - bx2 + ax2
f = d * ax2 + e * ay2
Det = e * a - b * d
x, y = (c * e - b * f)/Det, (a * f - c * d) / Det
rows.append({
    "psi1 (deg)": psi1,
    "psi2 (deg)": psi2,
    "x (cm)": x.nominal_value, "s(x) (cm)": x.std_dev, "y (cm)": y.
↪nominal_value, "s(y) (cm)": y.std_dev
})
X_points.append(((c * e - b * f)/Det, (a * f - c * d) / Det))

X_table = pd.DataFrame.from_records(rows)

print("Estimated crossing of lines through detectors")
print(X_table.to_string(index = False, float_format = "{:.2f}".format))

```

Estimated crossing of lines through detectors

psi1 (deg)	psi2 (deg)	x (cm)	s(x) (cm)	y (cm)	s(y) (cm)
90	0	-1.68	0.07	2.69	0.09

Úkol: Zkonstruujte konfidenční elipsu pro souřadnice průsečíku (jedná se o pár náhodných proměnných s nejistotami).

[14]: *# Solution.*

```

from matplotlib.patches import Ellipse
import matplotlib.transforms as transforms

def confidence_ellipse(x, y, ax, n_std=1.0, facecolor='none', **kwargs):
    """
    Create a plot of the confidence ellipse of *x* and *y*.
    :param x, y : uncertainties random variables (scalar)
    :param ax matplotlib.axes.Axes The Axes object to draw the ellipse into.
    :param n_std : float The number of standard deviations to determine the
    ↪ellipse's radiuses.
    :param **kwargs Forwarded to `~matplotlib.patches.Ellipse`
    :return matplotlib.patches.Ellipse The ellipse object.
    """

    cov = correlation_matrix([x, y])
    pearson = cov[0, 1]

```

```

# Using a special case to obtain the eigenvalues of this
# two-dimensional dataset.
ell_radius_x = np.sqrt(1 + pearson)
ell_radius_y = np.sqrt(1 - pearson)
ellipse = Ellipse((0, 0), width=ell_radius_x * 2, height=ell_radius_y * 2,
                  facecolor=facecolor, **kwargs)

# Calculate the standard deviation of x from the square root of the
↪variance and multiply
# with the given number of standard deviations.
scale_x = x.std_dev * n_std
mean_x = x.nominal_value

# calculate the standard deviation of y ...
scale_y = y.std_dev * n_std
mean_y = y.nominal_value

transf = transforms.Affine2D() \
    .rotate_deg(45) \
    .scale(scale_x, scale_y) \
    .translate(mean_x, mean_y)

ellipse.set_transform(transf + ax.transData)
return ax.add_patch(ellipse)

```

Úkol: Nakreslete lebku a vykreslete přímky spojující detektory A a B a jejich průsečík spolu s konfidenční elipsou.

```

[15]: # Solution.

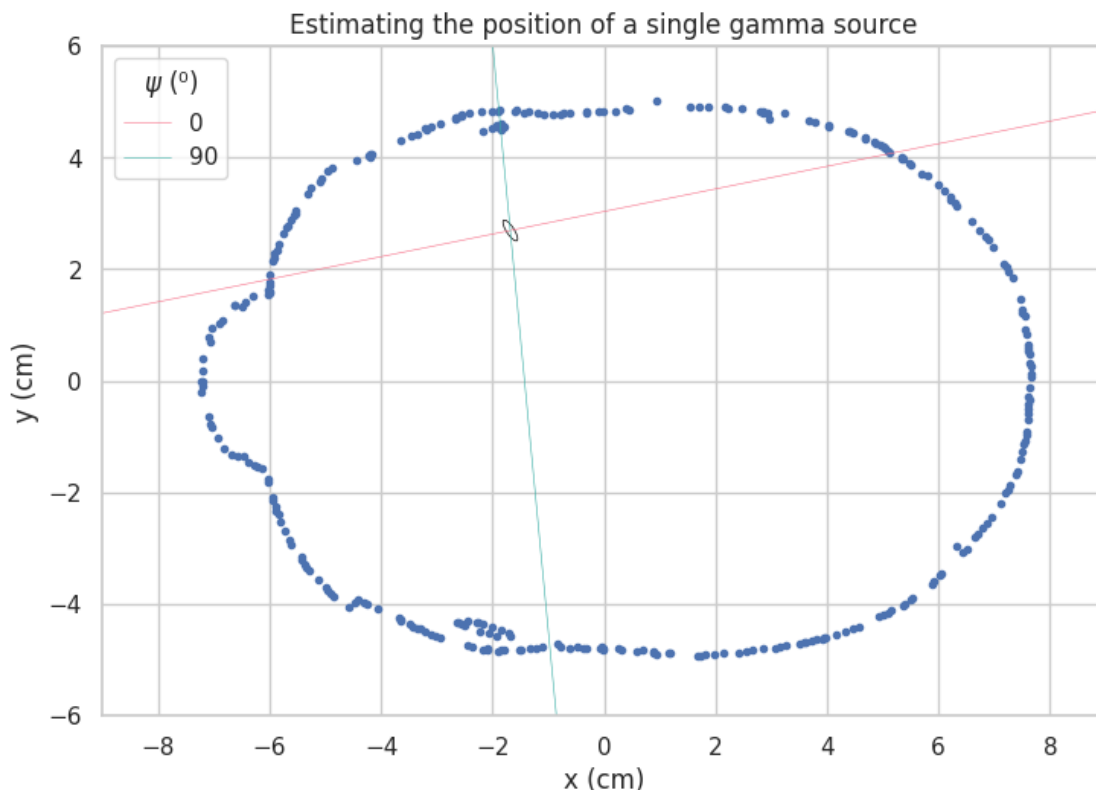
fig, ax = plt.subplots(figsize = [8.5, 8.5])
sns.lineplot(data = points_table, x = "x (cm)", y = "y (cm)", hue = "psi_
↪(deg)", linewidth = 0.4,
            palette = sns.color_palette("husl", len(psi_vals)))
ax.scatter(skull_data["x"], skull_data["y"], marker = ".")
confidence_ellipse(X_points[0][0], X_points[0][1], ax, n_std = 2,
                  edgecolor = "black", linewidth = 0.5)
ax.set_aspect("equal")
plt.xlabel("x (cm)")
plt.xlim([-9, 9])
plt.ylabel("y (cm)")
plt.ylim([-6,6])
plt.legend(title = "$\\psi$ ( )")
plt.title("Estimating the position of a single gamma source")

```

```

[15]: Text(0.5, 1.0, 'Estimating the position of a single gamma source')

```



Úkol: Diskutujte výhody a nevýhody použité metody na určení polohy zdroje pozitronů. Diskutujte její přesnost a možnosti jejího zlepšení. Odpovězte mj. na následující otázky:

- Jaká je přesnost měření? Můžeme se spolehnout na chyby fitovaných parametrů?
- Jak bychom mohli zlepšit přesnost? Co byste upřednostnili pro lepší odhad přesnosti měření:
 - Přidat další úhlové rozdělení (změřit při jiném ψ), nebo
 - Měřit úhlová rozdělení důkladněji (delší čas akumulace, menší úhlový krok)?

Řešení:

Mezi výhody této metody pro určení polohy zdroje pozitronů patří její jednoduchost a intuitivnost. Metoda vyžaduje jen několik měření a pouze základní geometrické výpočty. Také se jedná o rychlou vizualizaci dat - stačí určit průsečík přímk. S tím souvisí také nízké nároky na výpočetní výkon.

Naopak jako nevýhodu můžeme označit nízkou přesnost. Nejistota úhlu detektoru i fitu přímo způsobují nejistotu polohy průsečíku. Pro negaussovský profil závislosti již fitované parametry příliš neodpovídají a zvyšují tak nejistotu, konkrétně poloha maxima, kterou využíváme. Dalším omezením je měření více zdrojů než 2. Průsečíky se začnou komplikovat a je pravděpodobné, že peaky incidentů od jednotlivých zářičů se začnou přerývat pro různá natočení. Nakonec jako nejistotu považujeme odmocninu z počtu incidencí, tedy Poissonovo rozdělení a neyvačujeme vůbec systematické chyby jako poloha lebky nebo rozměry detektorů. To je zároveň odpověď na otázku, že se nemůžeme spolehnout na nejistoty fitovaných parametrů pro určení polohy.

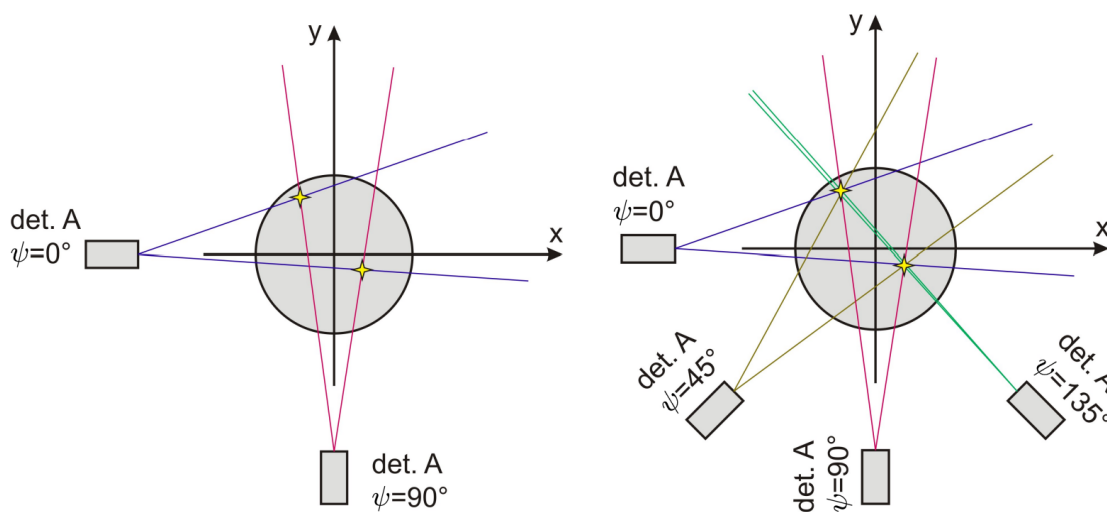
Co se týče zlepšení přesnosti prodloužením času akumulace a menšími úhlovými kroky bychom snížili chybu fitovaných parametrů, ale to jsme již, viz. předchozí diskuze, označili jako dobrý fit. Proto bych upřednostnil přidat další úhlové rozdělení, ze kterých získáme více přímk a snížili bychom tak vysokou chybovost nejistoty určení polohy.

3 Měření se dvěma pozitronovými zdroji

V případě N pozitronových zdrojů pozorujeme superpozici N gaussovských distribucí. Pro dva zdroje tedy očekáváme bimodální distribuci se dvěma maximy.

1. Některé čáry se mohou křížit mimo lebku a taková křížení lze bezpečně ignorovat.
2. Pokud mají zdroje odlišnou aktivitu, můžeme identifikovat vrcholy v úhlových rozloženích patřících jednotlivým zdrojům.
3. V tomto notebooku použijeme oblasti spolehlivosti pro křížení k identifikaci těch, které jsou shodné v rámci chyby měření.

Obr. 2 Pozitronová emisní tomografie: měření se dvěma pozitronovými zdroji, používané úhlové proměnné.



3.0.1 Postup měření

1. Poté, co instruktor umístí druhý pozitronový zdroj do modelu lebky, změřte úhlové rozdělení počtu koincidenzí v rozsahu úhlů ϕ a ψ potřebném k nalezení poloh obou zdrojů.
2. Pomocí pravítka určete přibližnou polohu obou zdrojů pozitronů.
3. Proveďte podrobnou analýzu pomocí tohoto notebooku.

3.0.2 Měření úhlových rozdělení

Úkol: Změřte úhlová rozdělení počtu koincidenzí pro tři různé úhly natočení lebky vzhledem ke stolu: $\psi_1 = 0 \text{ deg}$, $\psi_2 = 90 \text{ deg}$, $\psi_3 = 120 \text{ deg}$. Naměřená data pište do stringu `data_str` níže;

jeho formát je stejný jako v předchozí části.

```
[16]: # Solution.

# TODO: Fill the string below with your data.
# The string will look like the following, but with many more lines, of course:
# data_str = ""
# psi phi N
# 0    -35 5
# 0    -31 11
# 90   -35 6
# 90   -31 10
# 120  -35 6
# 120  -31 10
# ""

data_str = ""
psi phi N
0 -20 11
0 -15 30
0 -13 34
0 -10 66
0 -9 42
0 -8 62
0 -7 66
0 -6 66
0 -5 89
0 -4 91
0 -3 100
0 -2 101
0 -1 67
0 0 88
0 2 86
0 4 89
0 5 84
0 7 77
0 10 86
0 13 86
0 16 105
0 18 125
0 19 149
0 20 157
0 21 185
0 22 172
0 23 171
0 24 147
0 25 145
```


0 26 138
0 28 117
0 30 97
0 33 58
0 36 24
90 -40 25
90 -35 67
90 -32 90
90 -30 145
90 -29 141
90 -28 132
90 -27 141
90 -26 130
90 -25 161
90 -24 140
90 -23 130
90 -21 112
90 -18 56
90 0 97
90 3 104
90 7 133
90 8 150
90 9 148
90 10 159
90 11 162
90 12 146
90 13 160
90 14 146
90 15 125
90 17 141
90 19 114
90 22 76
90 25 58
130 -30 36
130 -25 142
130 -24 131
130 -23 171
130 -22 172
130 -21 187
130 -20 219
130 -19 220
130 -18 203
130 -17 267
130 -16 257
130 -15 216
130 -14 234
130 -13 224

```

130 -12 214
130 -11 216
130 -10 192
130 -7 148
130 -4 133
130 -1 74
130 2 44
"""

```

Úkol: Ze stringu obsahujícího vaše data vytvořte pandas DataFrame `data_2`. Do něj přidejte také sloupec `err`, který bude obsahovat neurčitosti počtu koincidencí, které lze odhadnout jako $\sqrt{N}$. DataFrame `data_2` vypište jako tabulku.

```

[17]: # Solution.

from io import StringIO

# Create the pandas dataframe from the string above.
data_2 = pd.read_table(StringIO(data_str), sep = "\s+")

# Add a column with uncertainties in counts.
data_2["err"] = np.sqrt(data_2["N"])

# Print data_2 as a table.
print(data_2.rename(columns = {"psi": "psi [deg]", "phi": "phi [deg]", "N": "N",
    ↪ "N", "err": "N_unc"}).to_string(index = False, max_rows = None))

# # ----- SAFE TO IGNORE BELOW THIS LINE -----
# # ALTERNATIVE SOLUTION: Read the data from a file.
# # The file has the same format as the `data_str` string above: three columns
# # with a header line. The columns are separated by whitespace (spaces or
    ↪ tabs).
# # The header line is: "psi phi N".

# # Path to the data file.
# data_file = A7_home + "/sample_data/set_A/Task2.txt"

# # Read the data of a single source angular profile from a file and convert to
    ↪ a pandas dataframe
# data_2 = pd.read_table(data_file, sep = "\s+")
# print(data_2.head())
# data_2["err"] = np.sqrt(data_2["N"]) # compute uncertainties in counts

```

psi [deg]	phi [deg]	N	N_unc
0	-20	11	3.316625
0	-15	30	5.477226
0	-13	34	5.830952

0	-10	66	8.124038
0	-9	42	6.480741
0	-8	62	7.874008
0	-7	66	8.124038
0	-6	66	8.124038
0	-5	89	9.433981
0	-4	91	9.539392
0	-3	100	10.000000
0	-2	101	10.049876
0	-1	67	8.185353
0	0	88	9.380832
0	2	86	9.273618
0	4	89	9.433981
0	5	84	9.165151
0	7	77	8.774964
0	10	86	9.273618
0	13	86	9.273618
0	16	105	10.246951
0	18	125	11.180340
0	19	149	12.206556
0	20	157	12.529964
0	21	185	13.601471
0	22	172	13.114877
0	23	171	13.076697
0	24	147	12.124356
0	25	145	12.041595
0	26	138	11.747340
0	28	117	10.816654
0	30	97	9.848858
0	33	58	7.615773
0	36	24	4.898979
90	-40	25	5.000000
90	-35	67	8.185353
90	-32	90	9.486833
90	-30	145	12.041595
90	-29	141	11.874342
90	-28	132	11.489125
90	-27	141	11.874342
90	-26	130	11.401754
90	-25	161	12.688578
90	-24	140	11.832160
90	-23	130	11.401754
90	-21	112	10.583005
90	-18	56	7.483315
90	0	97	9.848858
90	3	104	10.198039
90	7	133	11.532563
90	8	150	12.247449

90	9	148	12.165525
90	10	159	12.609520
90	11	162	12.727922
90	12	146	12.083046
90	13	160	12.649111
90	14	146	12.083046
90	15	125	11.180340
90	17	141	11.874342
90	19	114	10.677078
90	22	76	8.717798
90	25	58	7.615773
130	-30	36	6.000000
130	-25	142	11.916375
130	-24	131	11.445523
130	-23	171	13.076697
130	-22	172	13.114877
130	-21	187	13.674794
130	-20	219	14.798649
130	-19	220	14.832397
130	-18	203	14.247807
130	-17	267	16.340135
130	-16	257	16.031220
130	-15	216	14.696938
130	-14	234	15.297059
130	-13	224	14.966630
130	-12	214	14.628739
130	-11	216	14.696938
130	-10	192	13.856406
130	-7	148	12.165525
130	-4	133	11.532563
130	-1	74	8.602325
130	2	44	6.633250

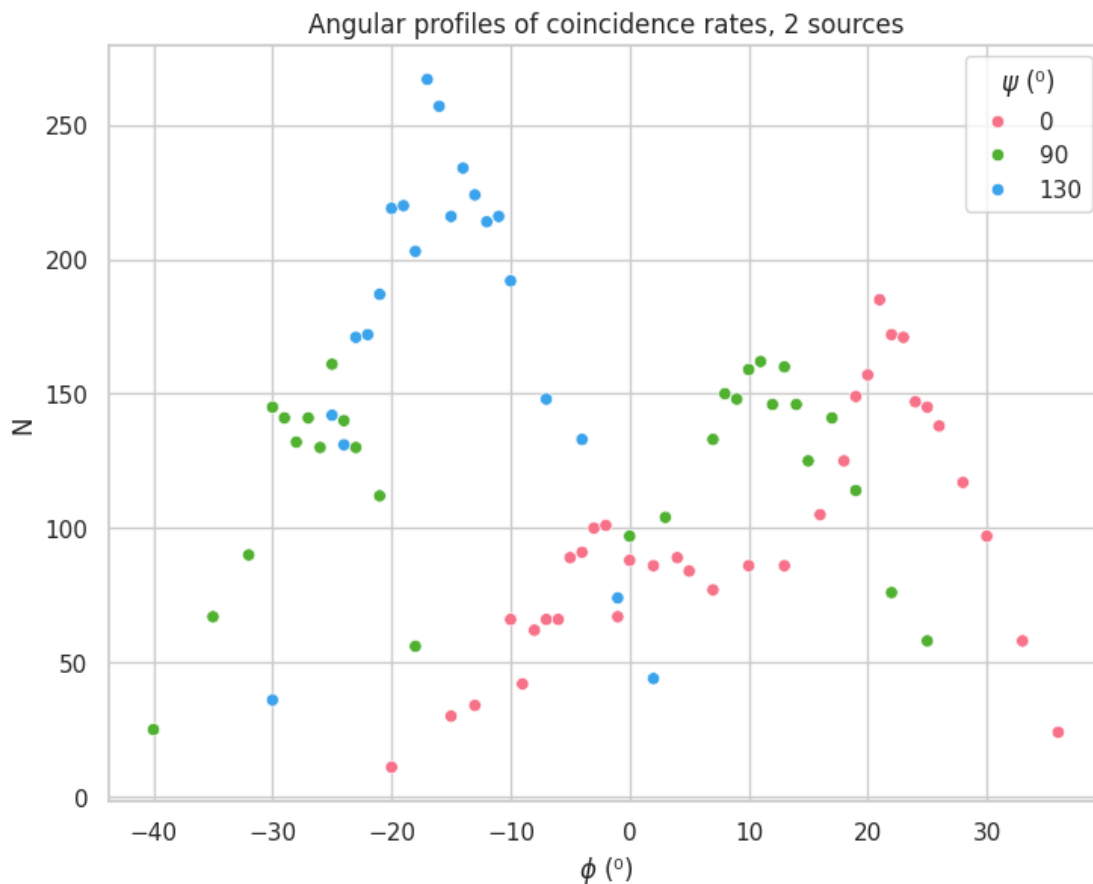
Úkol: Vykreslete naměřená data jako bodový graf četnosti koincidencí N v závislosti na úhlu ϕ pro všechny úhly ψ .

```
[18]: # Solution.

psi_vals = data_2["psi"].unique() # Replace psi with these values

fig, ax = plt.subplots(figsize=(8.5, 6.5))
sns.scatterplot(data = data_2, x = "phi", y = "N", hue = "psi", palette = sns.
    color_palette("husl", len(psi_vals)), ax = ax)
plt.xlabel("$\\phi$ ( )")
plt.legend(title = "$\\psi$ ( )")
plt.title("Angular profiles of coincidence rates, 2 sources")
```

```
[18]: Text(0.5, 1.0, 'Angular profiles of coincidence rates, 2 sources')
```



Úkol: Vysvětlete tvar naměřených úhlových rozdělení počtu koincidencí.

Řešení:

Uvnitř lebky jsou nyní již umístěny dva zářiče s rozdílnou intenzitou koincidence (to znamená aktivitou nebo vzdáleností). Tomu odpovídá tvar naměřených úhlových rozdělení. Pozorujeme u dvou závislostí dva peaky pro dva tyto zářiče a pro modrý graf pozorujeme překryv zářičů. Proto vidíme závislost připomínající multiplet a hodnoty koincidenčí jsou vyšší než pro ostatní měření.

Závislost jednotlivých peaků je pyramidová, to znamená, že koincidence lineárně roste a po určitém úhlu lineárně klesá.

3.0.3 Fitování dat

Nyní určíme polohy maxim naměřených úhlových rozdělení. Budeme je odhadovat pomocí fitování. Fity jsou nyní ovšem poněkud komplikované, protože někdy máme jasně bimodální distribuci a někdy jsou oba zdroje blízko sebe a distribuce vypadá jako jediný široký peak. V takovém případě budeme muset fity nastavit ručně.

Pokud některý fit vyjde špatně, podívejte se na graf naměřených úhlových rozdělení a upravte počáteční podmínky tak, aby odpovídaly tomu, co v něm vidíte.

Úkol: Na každé úhlové rozdělení nafitujte funkci, která je součtem dvou normálních rozdělení a konstanty. Nejprve nastavte počáteční podmínky pro každý fit podle toho, co vidíte v grafu naměřených dat. Počáteční podmínky definujte v stringu `initial_guess_str` níže. O hodnoty úhlu ψ se nestarejte, ty budou načteny z DataFrame `data_2` později; prozatím použijte libovolné hodnoty. Pokud chcete, aby fitovaná funkce pro některé rozdělení obsahovala pouze jeden gaussián, nastavte amplitudu druhého gaussiánu `amp2` na nulu. Výsledky fitů uveďte v tabulce. “

```
[19]: # Solution.

# TODO: Replace values in the string `initial_guess_str` with your initial
#       guesses for fit parameters.
initial_guess_str = '''
psi bg amp1 mean1 width1 amp2 mean2 width2
psi1 5 100 -15 6 100 5 6
psi1 5 100 -5 6 100 5 6
psi1 5 100 -15 6 0 5 6
'''

psi_vals = data_2["psi"].unique() # Replace psi with these values
initial_guess = pd.read_csv(StringIO(initial_guess_str), sep = "\s+")
initial_guess["psi"] = psi_vals
initial_guess = initial_guess.set_index("psi")
initial_guess = initial_guess.sort_index() # Sorting the DataFrame by the
#       index (psi) for clarity
initial_guess = initial_guess.reset_index()
initial_guess.style.format(precision=2).set_caption("Parameters of fits to
#       angular profiles of coincidence rates")

# -----
# Fitting.
# -----

def double_gauss_plus_background(x, bg, amp1, mean1, sig1, amp2, mean2, sig2):
    """
    Function returning a double gaussian with flat background,
    Meaning of parameters is clear from the definition.
    """
    return bg + amp1 * np.exp(- 1/2 * ((x-mean1)/sig1)**2) + amp2 * np.exp(- 1/
#       2 * ((x-mean2)/sig2)**2)

fits = dict.fromkeys(psi_vals)
for psi in psi_vals:
    data_psi = data_2[data_2["psi"] == psi]
    mid = np.sum(data_psi["phi"] * data_psi["N"]) / np.sum(data_psi["N"])
    top = max(data_psi["N"])
```

```

    bounds = ([0, 0, -np.inf, 0, 0, -np.inf, 0], [np.inf, np.inf, np.inf, np.
↳inf, np.inf, np.inf, np.inf])
    x_scale = (5, 200, 20, 7, 200, 20, 7)
    initial_guess_psi = initial_guess[initial_guess["psi"] == psi].values[0][1:]
    if initial_guess_psi[4] != 0: # Two component fit is requested
        try:
            p_opt, p_cov = curve_fit(double_gauss_plus_background,
↳data_psi["phi"].values, data_psi["N"].values,
                                p0 = initial_guess_psi,
                                sigma = data_psi["err"].values, absolute_sigma = True,
↳bounds = bounds, x_scale = x_scale)
        except (RuntimeError, ValueError, OptimizeWarning) as e:
            print(e)
            print("Reverting to single fit for psi = ", psi)
            bounds = (bounds[0][:4], bounds[1][:4])
            x_scale = x_scale[:4]
            p_opt, p_cov = curve_fit(gauss_plus_background, data_psi["phi"].
↳values, data_psi["N"].values,
                                p0 = initial_guess_psi[:4],
                                sigma = data_psi["err"].values, absolute_sigma = True)
        else:
            bounds = (bounds[0][:4], bounds[1][:4])
            x_scale = x_scale[:4]
            p_opt, p_cov = curve_fit(gauss_plus_background, data_psi["phi"].values,
↳data_psi["N"].values,
                                p0 = initial_guess_psi[:4],
                                sigma = data_psi["err"].values, absolute_sigma = True)

    p_err = np.sqrt(np.diag(p_cov))
    errXerr = np.outer(p_err, p_err)
    p_corr = p_cov / errXerr
    if len(p_opt) == 4:
        p_opt = np.asarray([*p_opt, None, None, None])
        p_err = np.asarray([*p_err, None, None, None])
        fits[psi] = {"parameters": p_opt, "par_errors": p_err, "correlations" :
↳p_corr}

# No units in the header, since the table would be too broad.
fits_bg = pd.DataFrame({
    "psi" : psi_vals,
    "bg" : [fits[psi]["parameters"][0] for psi in psi_vals],
    "s(bg)" : [fits[psi]["par_errors"][0] for psi in psi_vals],
})
fits_1 = pd.DataFrame({
    "psi" : psi_vals,
    "amp1" : [fits[psi]["parameters"][1] for psi in psi_vals],

```

```

"s(amp1)" : [fits[psi]["par_errors"][1] for psi in psi_vals],
"mean1" : [fits[psi]["parameters"][2] for psi in psi_vals],
"s(mean1)" : [fits[psi]["par_errors"][2] for psi in psi_vals],
"width1" : [fits[psi]["parameters"][3] for psi in psi_vals],
"s(width1)" : [fits[psi]["par_errors"][3] for psi in psi_vals],
})
fits_2 = pd.DataFrame({
    "psi" : psi_vals,
    "amp2" : [fits[psi]["parameters"][4] for psi in psi_vals],
    "s(amp2)" : [fits[psi]["par_errors"][4] for psi in psi_vals],
    "mean2" : [fits[psi]["parameters"][5] for psi in psi_vals],
    "s(mean2)" : [fits[psi]["par_errors"][5] for psi in psi_vals],
    "width2" : [fits[psi]["parameters"][6] for psi in psi_vals],
    "s(width2)" : [fits[psi]["par_errors"][6] for psi in psi_vals]
})

print("Parameters of fits to angular profiles of coincidence rates")
for df in [fits_bg, fits_1, fits_2]:
    print('-----')
    print(df.to_string(index = False, float_format = "{:.2f}".format))

```

Parameters of fits to angular profiles of coincidence rates

```

-----
psi      bg  s(bg)
0       2.29  6.25
90      14.80  9.15
130     -50.32 39.20
-----

psi  amp1  s(amp1)  mean1  s(mean1)  width1  s(width1)
0   86.97    5.86   0.27    0.80    9.57    1.08
90  132.61    8.97 -26.48    0.30    5.93    0.56
130 284.69   36.77 -14.54    0.20   10.58    1.30
-----

psi  amp2  s(amp2)  mean2  s(mean2)  width2  s(width2)
0   155.55    5.96  23.00    0.47    6.65    0.36
90   138.82    9.80  10.79    0.45    9.33    0.70
130    NaN    NaN    NaN    NaN    NaN    NaN

```

Úkol: Do DataFrame `data_2` přidejte:

- sloupec `N_fit`, který bude obsahovat hodnoty fitované funkce v bodech ϕ ,
- sloupec `residual`, který bude obsahovat veličinu $(N - N_{\text{fit}})/\sqrt{N_{\text{fit}}}$, tzv. reziduály.

Do dictionary `fits` přidejte hodnotu χ^2 , `chisq` a počet stupňů volnosti, `df`, pro každý fit.

[20]: *# Solution.*

```
def add_fitted_to_data(data):
```



```

'''
Adds fitted values and residuals to data points.
Also adds chi2 and df to the fits dictionary.
'''

result = data.copy()
psi = np.mean(result["psi"])
parameters = fits[psi]["parameters"]
if parameters[4] is None:
    result["N_fit"] = gauss_plus_background(data["phi"], *parameters[:4])
else:
    result["N_fit"] = double_gauss_plus_background(data["phi"], *parameters)
result["residual"] = (result["N"] - result["N_fit"]) / np.
↳sqrt(result["N_fit"])
fits[psi]["chisq"] = sum(r**2 for r in result["residual"]) # side effect !
nrow, ncol = data.shape
fits[psi]["df"] = nrow - len(parameters)
return result

data_2 = data_2.groupby("psi")[["psi", "phi", "N", "err"]].
↳apply(add_fitted_to_data)
data_2.index = data_2.index.droplevel("psi")
print(data_2.head())

```

	psi	phi	N	err	N_fit	residual
0	0	-20	11	3.316625	11.521578	-0.153661
1	0	-15	30	5.477226	26.638212	0.651355
2	0	-13	34	5.830952	35.539207	-0.258192
3	0	-10	66	8.124038	51.179074	2.071711
4	0	-9	42	6.480741	56.683374	-1.950285

Úkol: Naměřená data a nafitované funkce zobrazte v jednom grafu.

[21]: # Solution.

```

def make_fine_plot(data):
    '''
    Data to draw a fitted function plot
    '''
    psi = np.mean(data["psi"]).astype(int)
    parameters = fits[psi]["parameters"]
    x = np.linspace(min(data["phi"]), max(data["phi"]), 100)
    if parameters[4] is None:
        y = gauss_plus_background(x, *parameters[:4])
    else:
        y = double_gauss_plus_background(x, *parameters)
    return pd.DataFrame({"psi": np.repeat(psi, 100), "phi": x, "N_fit": y})

fine_data = data_2.groupby("psi")[["psi", "phi", "N_fit"]].apply(make_fine_plot)

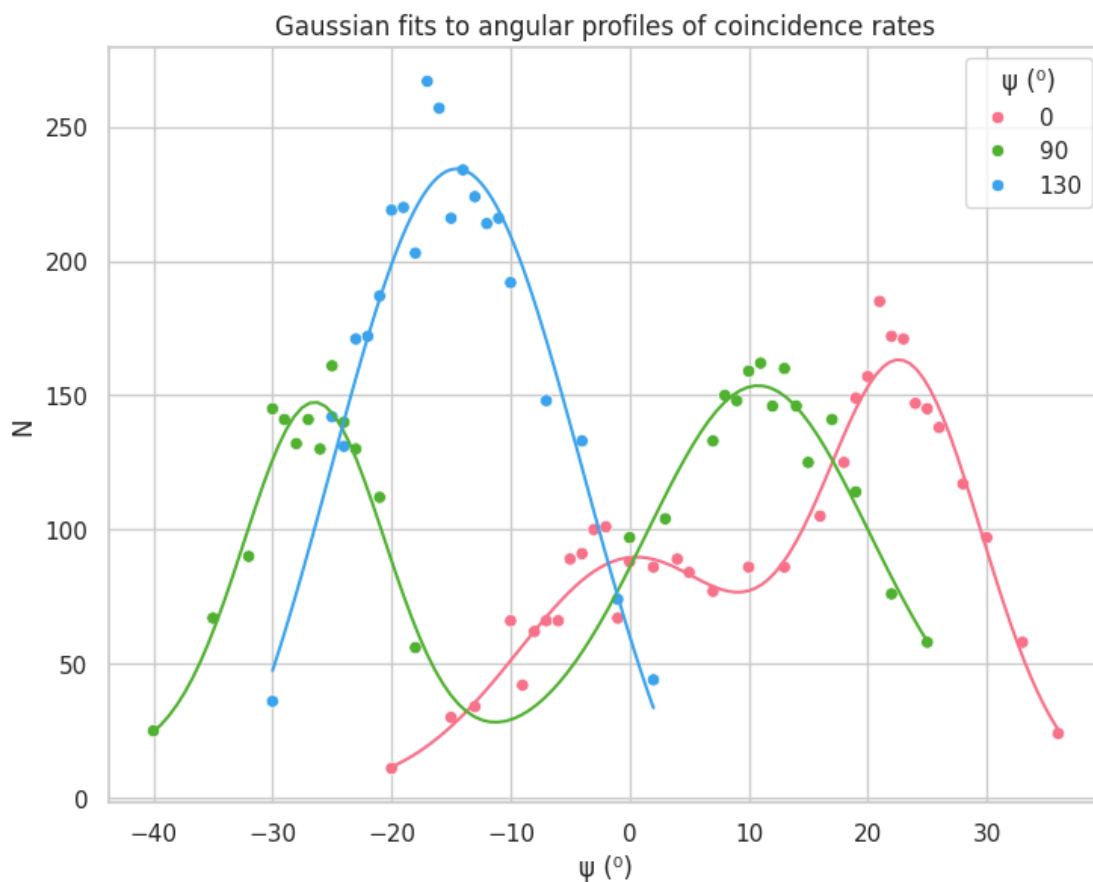
```

```

fig, ax = plt.subplots(figsize=(8.5, 6.5))
sns.scatterplot(data = data_2, x = "phi", y = "N", hue = "psi", palette = sns.
    color_palette("husl", len(psi_vals)), ax = ax)
sns.lineplot(data = fine_data, x = "phi", y = "N_fit", hue = "psi", palette =
    sns.color_palette("husl", len(psi_vals)), ax = ax, legend = False)
plt.xlabel(" ")
plt.legend(title = " ")
plt.title("Gaussian fits to angular profiles of coincidence rates")

```

[21]: Text(0.5, 1.0, 'Gaussian fits to angular profiles of coincidence rates')



3.0.4 Diagnostické grafy

Nyní se budeme zabývat otázkou, zda jsou naše fity rozumné - zda zvolený model odpovídá naměřeným datům.

Úkol: Pro každý fit vypište hodnoty χ^2 , počet stupňů volnosti a redukovaný χ^2 .

```
[22]: # Solution.

# Print the chi2 and df values.
for psi in data_2["psi"].unique():
    chi2 = fits[psi]["chisq"]
    df = fits[psi]["df"]
    print(f"psi = {psi:3.0f} deg: chi2 = {chi2:6.1f}, df = {df:2d}, chi2/df = {chi2/df:5.2f}")
```

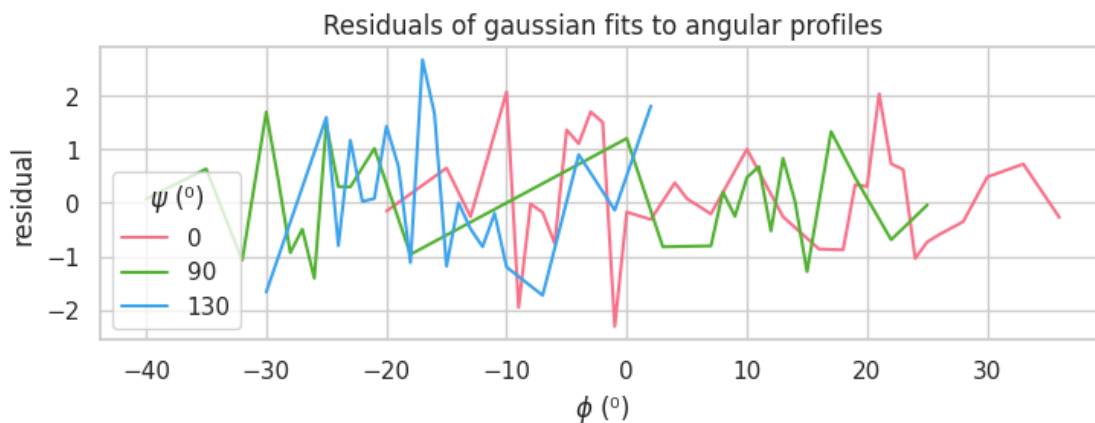
```
psi = 0 deg: chi2 = 33.8, df = 27, chi2/df = 1.25
psi = 90 deg: chi2 = 20.6, df = 21, chi2/df = 0.98
psi = 130 deg: chi2 = 31.8, df = 14, chi2/df = 2.27
```

Úkol: Do jednoho grafu nakreslete reziduály pro každé úhlové rozdělení.

```
[23]: # Solution.

fig, ax = plt.subplots(figsize = [8.5, 2.5])
sns.lineplot(data = data_2, x = "phi", y = "residual", hue = "psi", palette = "husl",
             dashes = True, ax = ax)
plt.xlabel("$\phi$ (°)")
plt.legend(title = "$\psi$ (°)")
plt.title("Residuals of gaussian fits to angular profiles")
```

```
[23]: Text(0.5, 1.0, 'Residuals of gaussian fits to angular profiles')
```



Úkol: Pro každé úhlové rozdělení nakreslete tzv. Quantile-Quantile (QQ) graf. Opět je cílem zjistit, jestli fitované funkce dobře popisují naměřená data.

```
[24]: def make_QQ_data(data):
    psi = np.mean(data["psi"]).astype(int)
    y = np.sort(data["residual"])
    mean, std = np.mean(y), np.std(y)
```

```

ppf = norm(loc=mean, scale=std).ppf # Inverse CDF
N = len(y)
x = [ppf(i / (N+2)) for i in range(1,N+1)]
nrow, ncol = data.shape
return pd.DataFrame({"psi": np.repeat(psi, nrow), "x": x, "y": y})

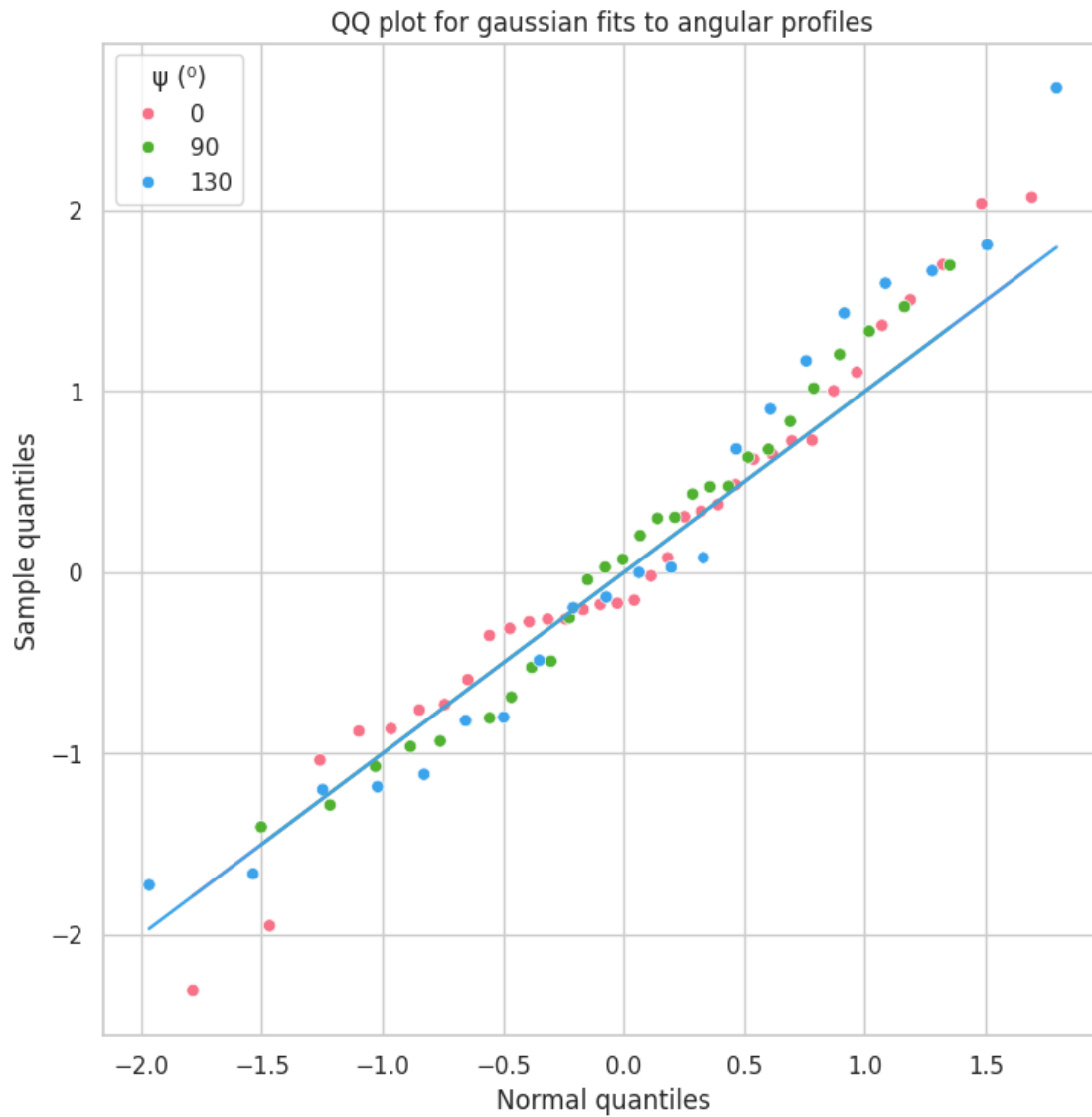
qq_data = data_2.groupby("psi")[["psi", "residual"]].apply(make_QQ_data)

fig, ax = plt.subplots(figsize = [8.5, 8.5])
sns.scatterplot(data = qq_data, x = "x", y = "y", hue = "psi", palette = sns.
    color_palette("husl", len(psi_vals)), ax = ax)
sns.lineplot(data = qq_data, x = "x", y = "x", hue = "psi", palette = sns.
    color_palette("husl", len(psi_vals)), ax = ax, legend = False)

# Add labels
plt.xlabel('Normal quantiles')
plt.ylabel('Sample quantiles')
plt.legend(title = " ( )")
plt.title("QQ plot for gaussian fits to angular profiles")

```

[24]: Text(0.5, 1.0, 'QQ plot for gaussian fits to angular profiles')



Úkol: Diskutujte kvalitu fitů a tvary naměřených úhlových rozdělání. Pokud je nějaký fit špatný, přenastavte pro něj počáteční hodnoty ve stringu `initial_guess_str` výše a vše spusťte znovu. Odpovězte mj. na následující otázky:

- Jsou provedené fity dobré? Mohly by být vylepšeny?
- Jsou profily gaussovské, nebo existuje podstatná odchylka od gaussovskosti? Proč?

Hint: dobrý fit by měl splňovat následující kritéria:

- Redukovaný χ^2 by měl být v rozumné blízkosti čísla 1.
- 95 % normalizovaných reziduálů by mělo být mezi -2 a 2.
- QQ graf by měl být přibližně lineární.

- standardní chyby parametrů by měly být rozumně nízké, tj. *ne* několikrát větší než hodnota parametru

V případě špatného fitu je obvyklým řešením použít jediný gaussian namísto dvou, pokud data neumožňují nafitovat součet dvou gaussianů.

Řešení:

Oproti měření s jedním zářičem vidíme, že fit je výrazně horší. Při porovnání redukovaného χ^2 dostáváme velmi dobrý výsledek pouze pro zelený graf $\psi = 90^\circ$, ostatní se již nenachází v rozumné blízkosti čísla 1. Požadavek na normalizované reziduály je splněn pro všechny fity, přesto ale pozorujeme větší oscilace. QQ graf je sice přibližně lineární, ale vizuálně vidíme, že přesnost je již horší než u předchozího měření. Celkově tato nasatvená parametrů odpovídá dobře, nedosahujeme ale tak vysoké přesnosti jako u měření jednoho zdroje.

Navíc pro nejlepší fit při $\psi = 90^\circ$ jsme museli natavit parametry počátečního odhadu, bez nichž fit neodpovídal. Stejně pro modrý graf bylo třeba nastavit fitování pouze jednou gaussovku, protože se zdroje překrývají.

Podobně jako v předchozí diskuzi, fitování dvojitou gaussovku bychom mohli vylepšit zjemněním kroku měření úhlu nebo zvýšit interval měření. Podstatnou odchylku od gaussovosti nepozorujeme.

3.0.5 Výpočet poloh pozitronových zdrojů

Začneme převedením parametrů fitu na (korelované) polohy zdrojů.

Poznámka: Dvě polohy detektoru B odpovídající dvěma gaussovským vrcholům jsou v textu i kódu níže označeny jako B a C.

Úkol: Vypočítejte polohy detektorů A a B v kartézské soustavě spojené s lebkou. Pro detektor B budete mít dvě polohy u rozdělení, která obsahovala dvě maxima. Výsledky vypište v tabulce.

```
[25]: # Solution.

# -----
# Convert fit parameters to random variables and set their correlations.
# This will help us propagate parameter uncertainties through calculations.
# -----

rv_fits = {} # contains line points as random variables
par_names = ["bg", "amp1", "mean1", "std1", "amp2", "mean2", "std2"]
for psi, fit in fits.items():
    rv_list = []
    for par, err in zip(fit["parameters"], fit["par_errors"]):
        if par:
            rv_list.append((par, err))
    rv_list = correlated_values_norm(rv_list, fit["correlations"])
    rv_fits[psi] = {par_names[i]: rv_list[i] for i in range(len(rv_list))}

# -----
```

```

# Get the positions of detectors A and B.
# -----

points = {} # to collect random variables
records = [] # to collect data for a table

for psi, par in rv_fits.items():
    # Order here is CAB or AB, depending on whether we have two peaks or one
    # Simplifies line plotting.
    try:
        ax, ay, cx, cy = to_xy(R, psi, par["mean2"])
        records.append({
            "psi (deg)" : psi,
            "ABC" : "C",
            "x (cm)" : cx.nominal_value,
            "s(x) (cm)" : cx.std_dev,
            "y (cm)" : cy.nominal_value,
            "s(y) (cm)" : cy.std_dev
        })
        points[psi] = {"C": (cx, cy)}
        C_str = f"C: x={cx.nominal_value:.2f}+/-{cx.std_dev:.2f}, y={cy.
↪nominal_value:.2f}+/-{cy.std_dev:.2f}"
    except KeyError:
        points[psi] = {}
        C_str = ""
    ax, ay, bx, by = to_xy(R, psi, par["mean1"])
    records.append({
        "psi (deg)" : psi,
        "ABC" : "A",
        "x (cm)" : ax.nominal_value,
        "s(x) (cm)" : ax.std_dev,
        "y (cm)" : ay.nominal_value,
        "s(y) (cm)" : ay.std_dev
    })
    records.append({
        "psi (deg)" : psi,
        "ABC" : "B",
        "x (cm)" : bx.nominal_value,
        "s(x) (cm)" : bx.std_dev,
        "y (cm)" : by.nominal_value,
        "s(y) (cm)" : by.std_dev
    })
    points[psi].update({"A": (ax, ay), "B": (bx, by)})

points_table = pd.DataFrame.from_records(records)
print("Estimated positions of detectors A and B/C")

```

```
print(points_table.sort_values(by = ["psi (deg)", "ABC"]).to_string(index =  
↪False, float_format = "{:.2f}".format))
```

Estimated positions of detectors A and B/C

psi (deg)	ABC	x (cm)	s(x) (cm)	y (cm)	s(y) (cm)
0	A	-15.00	0.50	-0.00	0.00
0	B	15.00	0.50	0.07	0.21
0	C	13.81	0.46	5.86	0.23
90	A	-0.00	0.00	-15.00	0.50
90	B	6.69	0.23	13.43	0.45
90	C	-2.81	0.15	14.73	0.49
130	A	9.64	0.32	-11.49	0.38
130	B	-6.45	0.22	13.54	0.45

Úkol: Pro každý úhel natočení lebky ψ a každou polohu detektoru B (B a C) zkonstruuje rovnice přímků spojujících detektory A a B v kartézské soustavě spojené s lebkou. Nalezněte průsečíky těchto přímků a vypište je v tabulce spolu s jejich neurčitostmi.

```
[26]: # Solution.  
  
# lines connecting AB and AC for each psi  
lines = {}  
for psi in psi_vals:  
    ax1, ay1 = points[psi]["A"]  
    bx1, by1 = points[psi]["B"]  
    a = by1 - ay1  
    b = - bx1 + ax1  
    c = a * ax1 + b * ay1  
    lines[psi] = {"AB" : (a, b, c)}  
    try:  
        cx1, cy1 = points[psi]["C"]  
    except KeyError:  
        continue  
    aa = cy1 - ay1  
    bb = - cx1 + ax1  
    cc = aa * ax1 + bb * ay1  
    lines[psi]["AC"] = (aa, bb, cc)  
  
# X-points of pairs of lines: size-2 combinations of all psi measurements  
from itertools import product  
rows = []  
X_points = []  
for psi1, psi2 in combinations(psi_vals, 2):  
    for bc1, bc2 in product("BC", repeat=2):  
        try: # some 2G fits may have failed  
            ax1, ay1 = points[psi1]["A"]  
            bx1, by1 = points[psi1][bc1]  
            ax2, ay2 = points[psi2]["A"]
```



```

        bx2, by2 = points[psi2][bc2]
    except KeyError:
        continue
    a = by1 - ay1
    b = - bx1 + ax1
    c = a * ax1 + b * ay1
    d = by2 - ay2
    e = - bx2 + ax2
    f = d * ax2 + e * ay2
    Det = e * a - b * d
    x = (c * e - b * f) / Det
    y = (a * f - c * d) / Det
    rho = correlation_matrix([x,y])[0,1]
    rows.append({
        "psi1 (deg)": psi1,
        "BC1": bc1,
        "psi2 (deg)": psi2,
        "BC2": bc2,
        "x (cm)" : x.nominal_value,
        "s(x) (cm)" : x.std_dev,
        "y (cm)" : y.nominal_value,
        "s(y) (cm)" : y.std_dev,
        "rho" : rho
    })
    X_points.append((x, y))

```

```

X_table = pd.DataFrame.from_records(rows)
print("Estimated crossing of lines through detectors")
print(X_table.to_string(index = False, float_format = "{:.2f}".format))

```

Estimated crossing of lines through detectors

psi1 (deg)	BC1	psi2 (deg)	BC2	x (cm)	s(x) (cm)	y (cm)	s(y) (cm)	rho
0	B	90	B	3.54	0.13	0.04	0.13	0.25
0	B	90	C	-1.42	0.08	0.03	0.09	-0.12
0	C	90	B	4.46	0.16	3.96	0.16	0.87
0	C	90	C	-1.67	0.09	2.71	0.11	-0.46
0	B	130	B	2.23	0.11	0.04	0.12	-0.69
0	C	130	B	0.26	0.05	3.10	0.12	-0.17
90	B	130	B	3.19	0.11	-1.45	0.07	-0.79
90	C	130	B	-2.05	0.12	6.70	0.29	-0.91

Úkol: Nakreslete lebku a vykreslete přímky spojující detektory A a B a jejich průsečíky.

[27]: # Solution.

```

fig, ax = plt.subplots(figsize = [8.5, 8.5])
sns.lineplot(data = points_table, x = "x (cm)", y = "y (cm)", hue = "psi (deg)",
             sort = False, palette = sns.color_palette("husl", len(psi_vals)),

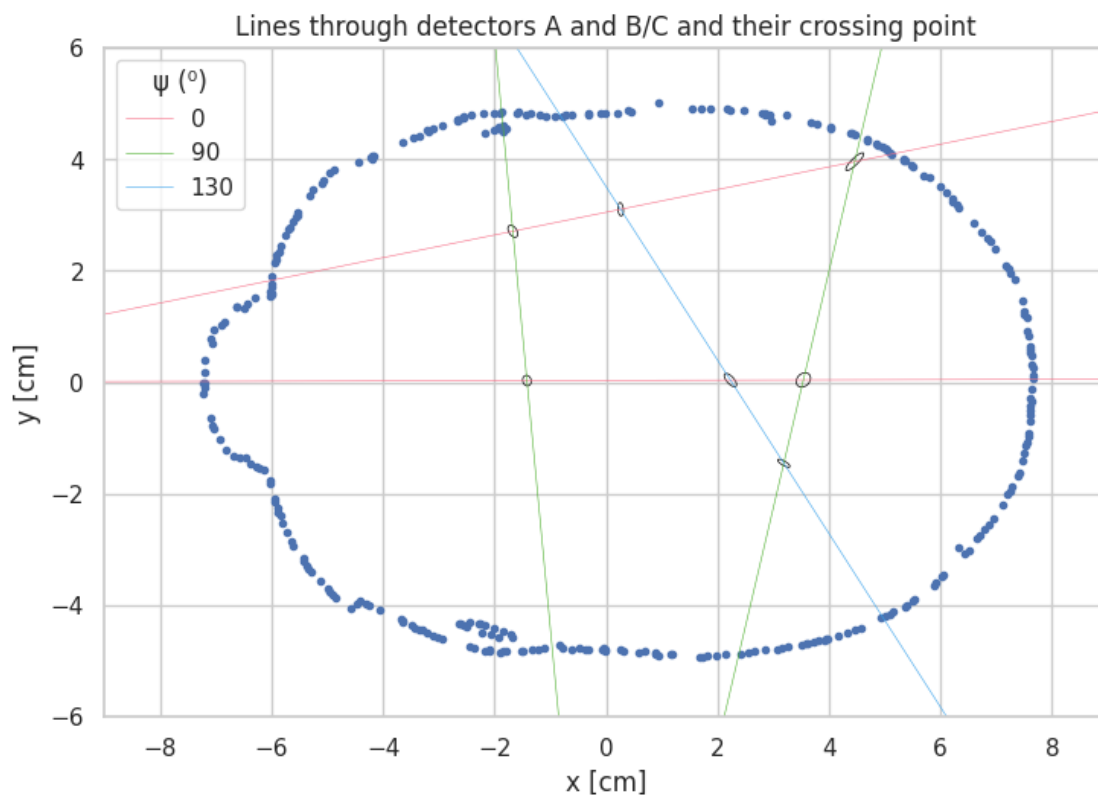
```

```

        linewidth = 0.4)
ax.scatter(skull_data["x"], skull_data["y"], marker = ".")
cmap = mpl.colormaps["coolwarm"]
colors = cmap(np.linspace(0, 1, len(X_points)))
for xy, color in zip(X_points, colors):
    x, y = xy
    confidence_ellipse(x, y, ax, n_std = 1, edgecolor = "black", linewidth = 0.
↪5)
ax.set_aspect("equal")
plt.xlabel("x [cm]")
plt.xlim([-9, 9])
plt.ylabel("y [cm]")
plt.ylim([-6,6])
plt.legend(title = " ( )")
plt.title("Lines through detectors A and B/C and their crossing point")

```

[27]: Text(0.5, 1.0, 'Lines through detectors A and B/C and their crossing point')



Úkol: Odhadněte souřadnice obou zdrojů pozitronů v souřadné soustavě lebky. Navrhněte vhodný způsob, jak toho docílit. Diskutujte vámi zvolený postup, jeho výsledky a přesnost.

Poznámky:

- Pravděpodobně nevidíte přesné trojcestné křižovatky, ale přímky by se měly křížit v malé oblasti. Rozhodněte se, které přímky odpovídají kterému zdroji, a určete polohy obou zdrojů.
- Číselné hodnoty souřadnic všech průsečíků a jejich neurčitosti naleznete v tabulce výše.
- Proveďte váš odhad, přestože metoda Radonovy transformace pravděpodobně poskytne odhad lepší - na konci budete muset oba výsledky porovnat.
- Pokud v obrázku výše vidíte velmi velké elipsy spolehlivosti, nejpravděpodobnější příčinou je, že jste fitovali součet dvou gaussianů na data, na která to není vhodné. Vraťte se zpět a nastavte počáteční parametry tak, aby se na rozdělení, na kterém je vidět jen jeden gaussian, fitoval skutečně jen jeden.

3.0.6 Radonova transformace

Radonova transformace funkce $g(x, y)$ definované na 2D rovině je

$$g(p, \tau) = \int_{E^2} g(x, y) \delta(y - px - \tau) dx dy$$

$g(p, \tau)$ je integrál funkce $g(x, y)$ podél přímky $y = px + \tau$.

Radonova transformace je lineární zobrazení. Použijeme dvě důležité vlastnosti Radonovy transformace:

1. Obraz bodového zdroje v bodě (x^*, y^*) , tj. $g(x, y) = \delta(x - x^*)\delta(y - y^*)$, je přímka $\tau = -x^*p + y^*$ v Radonově rovině, tj. přímka se sklonem $-x^*$ a průsečíkem y^* .

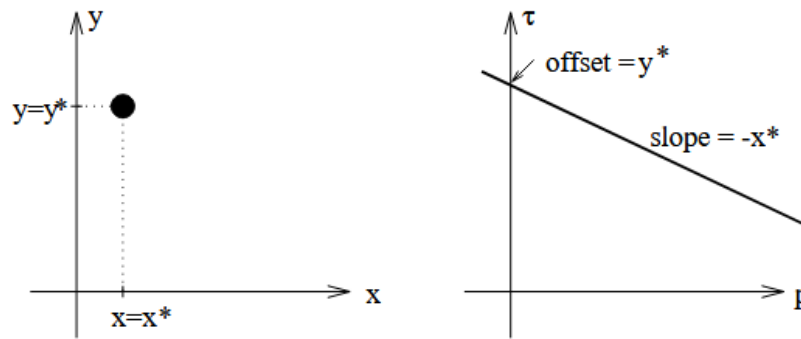
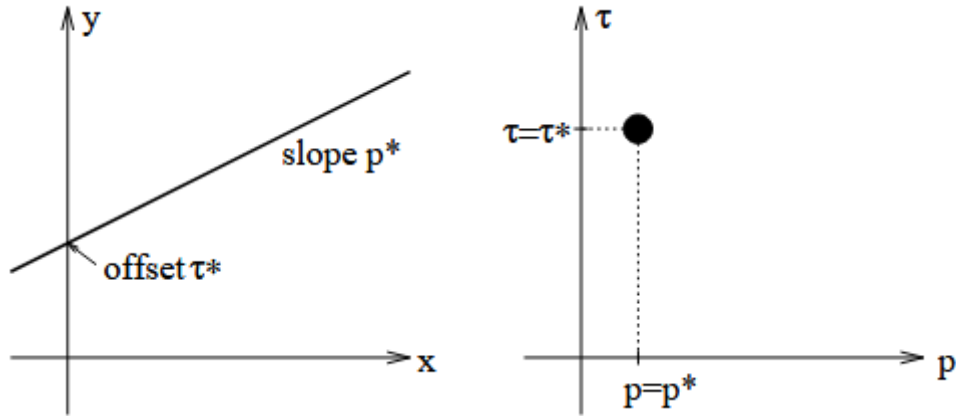


Figure 1.3 Left: A two dimensional function that only is non-zero in the point $(x, y) = (x^*, y^*)$. Right: The corresponding Radon transform (slant stacking result). Only when the Radon domain parameters matches the parameters of the line a non-zero result is found.

2. Obraz přímky $y = p^*x + \tau^*$ v E^2 je bod (p^*, τ^*) v Radonově rovině.



Obě vlastnosti lze snadno dokázat z definice.

V úkolech výše jsme rekonstruovali několik přímek mezi detektory A, B. Jejich Radonovy transformace budou body a lze je snadno vypočítat.

Pokud existují dva zdroje, body budou ležet na dvou přímkách - obrazech dvou bodů v E^2 představujících polohy obou zdrojů.

Jelikož nemusí být zřejmé, která dvojice přímek je správná, vyzkoušíme všechny kandidáty a vybereme dvojici, která poskytuje nejlepší fit metodou nejmenších čtverců. Fit implementujeme sami, abychom mohli použít nejistoty náhodných proměnných a získat dobré odhady chyb pro naše výsledky. Implementace je níže.

```
[28]: def least_squares_solution(a1, b1, c1, a2, b2, c2, a3, b3, c3):
    """
    Given 3 lines,  $a1*x + b1*y = c1$ ,  $a2*x + b2*y = c2$ ,  $a3*x + b3*y = c3$ , find
    their intersection
    in the least-squares sense - that is, solve the system of 3 equations for
    the unknowns  $x, y$ .
    :param a1, ... c3 scalars, coefficients of the 3 equations
    :return (x,y) Least-squares estimates of  $x, y$ .
    Note: no least-squares routine is called so that  $a1, \dots c3$  can be
    uncertainties random variables.
    Note: this only works with 3 lines, that is, measurements from 3 psi values.
    """
    # Normalize (neutral operation)
    n1 = umath.sqrt(a1**2 + b1**2)
    a1 = a1 / n1
    b1 = b1 / n1
    c1 = c1 / n1
    n2 = umath.sqrt(a2**2 + b2**2)
    a2 = a2 / n2
    b2 = b2 / n2
    c2 = c2 / n2
    n3 = umath.sqrt(a3**2 + b3**2)
```

```

a3 = a3 / n3
b3 = b3 / n3
c3 = c3 / n3

# Compute components of AtA
M11 = a1**2 + a2**2 + a3**2
M12 = a1*b1 + a2*b2 + a3*b3
M21 = M12 # Symmetric matrix
M22 = b1**2 + b2**2 + b3**2

# Compute determinant of AtA
detM = M11 * M22 - M12 * M21

# Compute Atc
v1 = a1*c1 + a2*c2 + a3*c3
v2 = b1*c1 + b2*c2 + b3*c3

# Compute least-squares solution
x = (M22 * v1 - M12 * v2) / detM
y = (-M21 * v1 + M11 * v2) / detM

chi2 = (c1 - a1*x - b1*y)**2 + (c2 - a2*x - b2*y)**2 + (c3 - a3*x - b3*y)**2

return x, y, chi2

```

Úkol: Sestavte všechny alternativní kombinace bodů/přímek, které reprezentují dva zdroje, a vyberte ty, které dávají nejlepší shodu.

```

[29]: # Solution.

from itertools import permutations

contents = [] # Collecting keys A,B/C of lines for each psi
for psi in psi_vals:
    contents.append(list(lines[psi].keys()))
max_content = len(max(contents, key = len))
slices = [None] * len(psi_vals) # Collecting slices for alternative sets of pq
    ↳ lines
slices[0] = [contents[0]] # No permutation for the first psi
for i_psi in range(1, len(psi_vals)):
    if len(contents[i_psi]) == max_content:
        slices[i_psi] = list(permutations(contents[i_psi]))
    else:
        slices[i_psi] = list(product(contents[i_psi], repeat = max_content)) #
    ↳ if not enough values use product
print("Slices:")

```

```

print(slices)

pq_lines = [list(zip(*slice)) for slice in list(product(*slices))] # Combine
↳ slices to sets of lines
print("Possible sets of Hough space lines:")
print(pq_lines)

# Find the set of lines with minimum chi2
min_set_chi2 = np.inf # minimum sum chi2 found for a set of lines
min_set_points = None
for ils, line_set in enumerate(pq_lines):
    total_chi2 = 0
    point_set = []
    for line in line_set:
        pq_points = []
        for i, psi in enumerate(psi_vals):
            a, b, c = lines[psi][line[i]]
            p = -a/b # Radon transform the line
            q = c/b
            pq_points.extend([p, 1, q])
        slope, intercept, chi2 = least_squares_solution(*pq_points) # fit line
↳ in pq space
        total_chi2 += chi2 # chi2 as rv, we take mean later
        xy_point = (-slope, intercept) # back to xy space
        point_set.append((xy_point, chi2))
    if total_chi2.nominal_value < min_set_chi2:
        min_set_chi2 = total_chi2.nominal_value
        min_set_points = point_set.copy()
print("Minimum chi2 set:")
print(f"{min_set_chi2}")
for x, y, chi2 in min_set_points:
    print(f"{x.nominal_value:.2f} +/- {x.std_dev:.2f}, {y.nominal_value:.2f} +/-
↳ {y.std_dev:.2f}, {chi2.nominal_value:.2f}")

```

Slices:

```
[[('AB', 'AC')], [('AB', 'AC'), ('AC', 'AB')], [('AB', 'AB')]]
```

Possible sets of Hough space lines:

```
[[('AB', 'AB', 'AB'), ('AC', 'AC', 'AB')], [('AB', 'AC', 'AB'), ('AC', 'AB', 'AB')]]
```

Minimum chi2 set:

```
min_set_chi2=2.147814405816864
```

```
3.03 +/- 0.11, -0.31 +/- 0.10, 0.53
```

```
-0.90 +/- 0.06, 3.28 +/- 0.12, 1.62
```

Úkol: Vše vykreslete a udělejte pěknou tabulku.

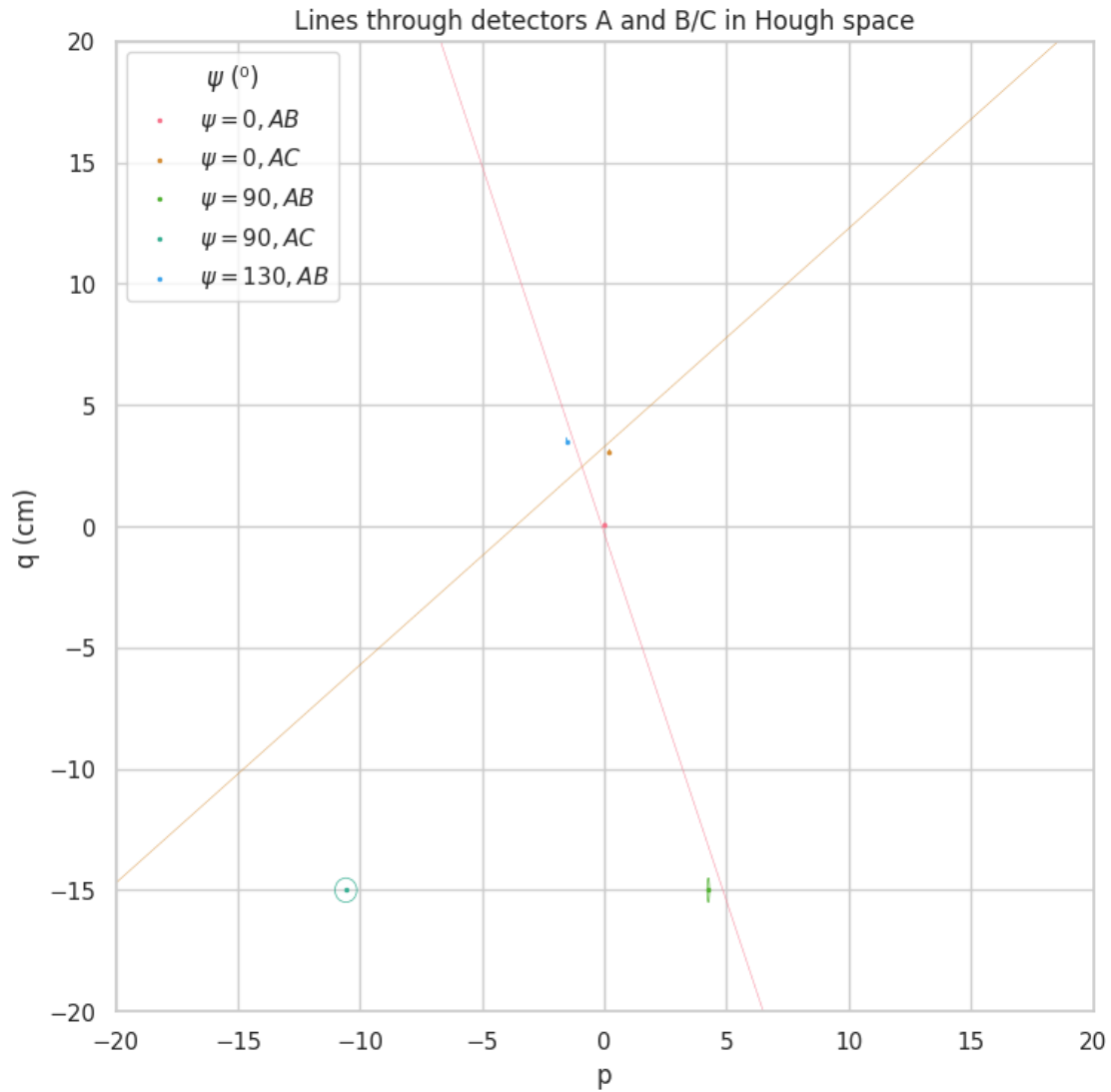
```

[30]: # Solution.

fig, ax = plt.subplots(figsize = [8.5, 8.5])
colors = sns.color_palette("husl", 3*len(psi_vals))
colors_1 = colors[::3]
colors_2 = colors[1::3]
for psi, color1, color2 in zip(psi_vals, colors_1, colors_2):
    a, b, c = lines[psi]["AB"]
    p = - a / b # slope, x-coordinates in Radon space
    tau = c / b # y-intercept, y-coordinate in Radon space
    plt.scatter([p.nominal_value], [tau.nominal_value], color = color1, s = 2,
    label = f"$\\psi = {psi}, AB$")
    confidence_ellipse(p, tau, ax, n_std = 1, edgecolor = color1, linewidth = 0.
    try:
        a, b, c = lines[psi]["AC"]
        p = - a / b
        tau = c / b
        plt.scatter([p.nominal_value], [tau.nominal_value], color = color2, s =
    label = f"$\\psi = {psi}, AC$")
        confidence_ellipse(p, tau, ax, n_std = 1, edgecolor = color2, linewidth
    except KeyError:
        continue
point_set_colors = sns.color_palette("husl", len(min_set_points))
for i, (x, y, chi2) in enumerate(min_set_points):
    slope = -x.nominal_value
    intercept = y.nominal_value
    plt.axline((0, intercept), slope = slope, linewidth = 0.3, color =
    #ax.set_aspect("equal")
plt.xlabel("p")
plt.xlim(-20, 20)
plt.ylabel("q (cm)")
plt.ylim(-20, 20)
plt.legend(title = "$\\psi$ ( )")
plt.title("Lines through detectors A and B/C in Hough space")

```

[30]: Text(0.5, 1.0, 'Lines through detectors A and B/C in Hough space')



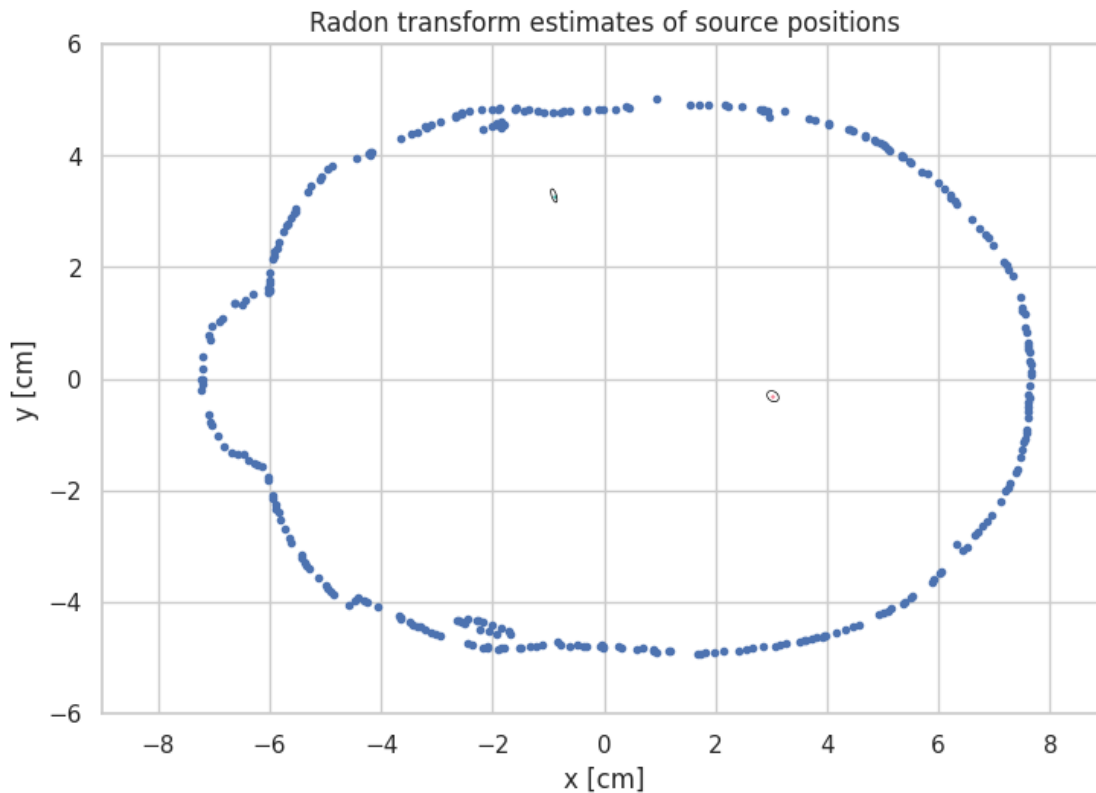
[31]: # Solution.

```
fig, ax = plt.subplots(figsize = [8.5, 8.5])
ax.scatter(skull_data["x"], skull_data["y"], marker = ".")
point_set_colors = sns.color_palette("husl", len(min_set_points))
for i, (x, y, chi2) in enumerate(min_set_points):
    plt.scatter([x.nominal_value], [y.nominal_value], s = 0.5, color = point_set_colors[i])
    confidence_ellipse(x, y, ax, n_std = 1, edgecolor = "black", linewidth = 0.5)
ax.set_aspect("equal")
plt.xlabel("x [cm]")
plt.xlim([-9, 9])
```



```
plt.ylabel("y [cm]")
plt.ylim([-6,6])
plt.title("Radon transform estimates of source positions")
```

[31]: Text(0.5, 1.0, 'Radon transform estimates of source positions')



[32]: # Solution.

```
rows = []
for x, y, chi2 in min_set_points:
    rho = correlation_matrix([x, y])[0,1]
    rows.append({
        "x (cm)" : x.nominal_value,
        "s(x) (cm)" : x.std_dev,
        "y (cm)" : y.nominal_value,
        "s(y) (cm)" : y.std_dev,
        "rho" : rho
    })
radon_table = pd.DataFrame.from_records(rows)
print("Estimated position of sources by Radon transform")
print(radon_table.to_string(index = False, float_format = "{:.2f}".format))
```

Estimated position of sources by Radon transform

x (cm)	s(x) (cm)	y (cm)	s(y) (cm)	rho
3.03	0.11	-0.31	0.10	-0.26
-0.90	0.06	3.28	0.12	-0.61

Úkol: Diskutujte provedené měření. Odpovězte mj. na následující otázky:

- Jaká je přesnost měření? Můžeme se spolehnout na neurčitosti parametrů fitů?
- Pokud byl první zdroj pozitronů ponechán na místě od prvního měření: jsou pozice z prvního a druhého měření konzistentní?
- Je výsledek konzistentní s odhadem průsečíků přímek založeným na rýsování na papír?
- Požádejte instruktora, aby pro vás otevřel model lebky. Pomocí pravítka změřte skutečné polohy pozitronových zdrojů. Odpovídají vaše měření realitě?
- Jak můžeme zlepšit přesnost/identifikaci? Co byste zvolili pro lepší odhad přesnosti měření:
 - Přidat další úhlové rozdělení (měření při jiném ψ), nebo
 - Měřit úhlová rozdělení důkladněji (delší doba akvizice, menší úhlový krok)?
 - nebo další zlepšení?
- Co kdybychom měli více zdrojů záření? Jaký je rozumný horní limit počtu zdrojů, jejichž polohy můžeme současně měřit?

Řešení:

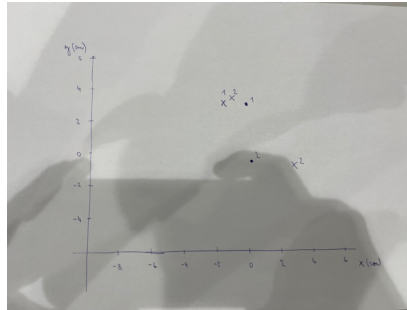
Při porovnání nejistot pro jeden zářič a dva zářiče dosahujeme vyšší spočtené přesnosti - $s(x)$ a $s(y)$ při prvním úkolu, tedy měření s jedním zdrojem. Podle dikuze výše jsme fitovali křivkami s nižší nejistotou, které ovlivňují výpočty poloh zářičů. Při měření dvou zdrojů jsme druhý pouze přidali a první zdroj nechali na stejném místě. Proto můžeme porovnat polohu jednoho zářiče při obou měřeních. Vidíme, že rozdíl těchto poloh je do 0,5 cm (v obou směrech), což se neshoduje v rámci spočtených chyb, ale přibližná poloha odpovídá. To navíc můžeme porovnat s ručně naměřenou polohou zářiče pomocí pravítka. Při tomto porovnání se poloha ve směru osy y shoduje, ve směru osy x se liší přibližně o 1 cm. Podobně jako v předchozím úkolu platí, že při změření více přímek bychom zvýšili přesnost této polohy. Neshodu dostáváme i pro druhý zářič, kde poloha na ose y přibližně odpovídá, na ose x se liší přibližně o 2 cm. Současně ale neuvažujeme nepřesnost měření pravítkem, kde se jedná spíše o odhad, protože díry jsou různě hluboké při různém sklonu a nelze tak jednoduše změřit polohy pravítkem.

Již pro naše měření dvou zdrojů jsme pozorovali překřížení dvou peaků pro určitý úhel, což jsme v našem případě využili pro výběr průsečíků a nebyl to problém. Pro více zdrojů by k takovému křížení docházelo mnohem častěji a byl by problém například rozlišit kolik zářičů se překrývá v případě zářičů s různou aktivitou a také naměření nepřekrývajících se peaků. Při vhodném rozložení zářičů v lebce by zřejmě bylo možné naměřit 3 zdroje.

Navíc jsme po doporučení vedoucím úlohy při měření navíc jako bonus znázornili výsledky ručně na papír. Křížky znázorňují dosažené výsledky měření s indexem 1 nebo 2 odpovídající číslu úkolu. Tečky odpovídají hodnotám naměřeným ručně pravítkem.

```
[33]: from IPython.display import Image
Image("IMG_6923.jpg")
```

[33]:



Navíc jsme proměřili koincidence pro konstantní úhly pro jeden zářič. Poloha zářiče zůstala stejná jako v prvním úkolu. Díky tomuto měření jsme proměřili pozadí. To může být způsobené náhodnými dvěma koincidence, které v rámci časového rozlišení detektoru způsobí, že dva fotony dorazí do obou detektorů v podobném čase z různých rozpadů. Dále také draslík přítomný v okolí způsobuje toto pozadí.

Dalším pozorovaným jevem je Comptonův rozptyl. Při přiblížení detektorů k sobě (snížení vzájemného úhlu natočení) docházelo ke zvýšení počtu koincidence. To je způsobeno Comptonovým rozptylem, tedy odražením fotonu z jednoho detektoru do druhého. To potvrzuje vložení olověné desky mezi detektory při malém úhlu, kde došlo opět ke snížení koincidence. Ostatní úhly můžeme použít pro odhad a potvrzení pozadí s dobou měření 100 s. Velikost úhlu je počítaná jako 0° pro detektory ležící v přímce a zvyšování úhlu odpovídá snižování vzájemného úhlu detektorů.

Naměřené hodnoty jsou uvedeny v tabulce níže.

```
[34]: import pandas as pd

# Data
df = pd.DataFrame({
    "Úhel [°]": [60, 90, 120, 120],
    "Koincidence": [17, 18, 27, 13],
    "Poznámka": ["", "", "", "s Pb"]
})

styled = (
    df.style
    .hide(axis="index")
    .set_properties(subset=["Úhel [°]", "Koincidence"], **{"text-align": "center"})
    .set_properties(subset=["Poznámka"], **{
        "text-align": "right",
        "color": "#666",
        "font-style": "italic",
        "white-space": "nowrap"
    })
)
```

```
    })  
)  
  
styled
```

[34]: <pandas.io.formats.style.Styler at 0x1482a8d89fa0>

Úkol: Sepište závěr.

Řešení:

4 Závěr

V úloze jsme experimentálně ověřili princip pozitronové emisní tomografie a určili polohu jednoho a dvou zdrojů pozitronů v modelu lebky. Pro jeden zdroj jsme z úhlových rozdělení při $\theta = 0^\circ$ a $\theta = 90^\circ$ získali maxima $22,9^\circ$ a $10,9^\circ$. Geometrická rekonstrukce dala polohu zdroje $x = -1,68 \pm 0,07$ cm, $y = 2,69 \pm 0,09$ cm, což se od skutečné polohy lišilo přibližně o 0,5–1 cm.

Při dvou zdrojích jsme pozorovali překrývání píků a museli použít dvojité Gaussovy fity. Nejlepší odhad poloh pomocí Radonovy transformace byl $(3,03 \pm 0,11$ cm; $-0,31 \pm 0,10$ cm) a $(-0,90 \pm 0,06$ cm; $3,28 \pm 0,12$ cm). Rozdíl oproti skutečné poloze byl do 1–2 cm, což je vzhledem k metodě přijatelné.

Přesnost byla omezená počtem úhlových měření (3 hodnoty θ) a krokem $\Delta\theta$. Zlepšení by přineslo více natočení lebky, jemnější krok ($0,5^\circ$) a delší dobu akumulace. Celkově metoda potvrdila princip PET, ale ukázala, že pro vyšší přesnost a více zdrojů je nutné použít pokročilejší rekonstrukční algoritmy.

5 Literatura

[1] P. Kvasnička, V. Vorobel et al., Studijní text k úloze A7, <https://physics.mff.cuni.cz/vyuka/zfp/zadani/407>, verze z 24.10.2022